

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
PHÂN HIỆU TRƯỜNG ĐẠI HỌC
GIAO THÔNG VẬN TẢI TẠI TP. HỒ CHÍ MINH
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



ĐỒ ÁN TỐT NGHIỆP

ĐỀ TÀI:

**HỆ THỐNG QUẢN TRỊ CƠ SỞ THỂ THAO THÔNG
MINH TÍCH HỢP GRAPH RAG VÀ AI AGENT**

Sinh viên thực hiện: NGUYỄN ĐỨC KHOA
MSSV: 6351071034
Lớp: Công Nghệ Thông Tin
Khóa: K63
Giảng viên hướng dẫn: ThS. Lưu Toàn Định

TP. Hồ Chí Minh Tháng 6 Năm 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
PHÂN HIỆU TRƯỜNG ĐẠI HỌC
GIAO THÔNG VẬN TẢI TẠI TP. HỒ CHÍ MINH
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



ĐỒ ÁN TỐT NGHIỆP

ĐỀ TÀI:

**HỆ THỐNG QUẢN TRỊ CƠ SỞ THỂ THAO THÔNG
MINH TÍCH HỢP GRAPH RAG VÀ AI AGENT**

Sinh viên thực hiện:	NGUYỄN ĐỨC KHOA
MSSV:	6351071034
Lớp:	Công Nghệ Thông Tin
Khóa:	K63
Giảng viên hướng dẫn:	ThS. Lưu Toàn Định

TP. Hồ Chí Minh Tháng 6 Năm 2026

MỤC LỤC

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN	i
LỜI MỞ ĐẦU	ii
DANH MỤC TỪ VIẾT TẮT	iii
DANH MỤC BẢNG	iv
DANH MỤC HÌNH ẢNH.....	vi
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI	1
1.1. Lý do chọn đề tài	1
1.2. Mục tiêu, nội dung và phương pháp nghiên cứu đề tài	1
1.2.1. Mục tiêu nghiên cứu	1
1.2.2. Nội dung nghiên cứu	2
1.2.3. Phương pháp nghiên cứu	3
1.3. Đối tượng và phạm vi nghiên cứu	3
1.3.1. Đối tượng nghiên cứu	3
1.3.2. Phạm vi nghiên cứu	3
1.4. Kết cấu báo cáo.....	4
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	5
2.1. API và giao tiếp giữa các thành phần	5
2.1.1. API (Application Programming Interface).....	5
2.1.2. Các giao thức thường dùng trong API.....	5
2.1.3. Ưu điểm của việc sử dụng API.....	5
2.2. Mô hình ngôn ngữ lớn và AI Agent	5
2.2.1. Mô hình ngôn ngữ lớn (LLM).....	5
2.2.2. Khái niệm AI Agent	7
2.2.3. Phân biệt chatbot LLM và AI Agent	8
2.3. Tool Calling và Function Calling	8
2.3.1. Khái niệm và cách gọi thuật ngữ	8
2.3.2. Schema và cơ chế sinh tham số	9
2.3.3. Luồng hoàn chỉnh của một tool call	9
2.3.4. LangChain và AgentExecutor	10
2.3.5. Prompt Engineering và System Prompt	10
2.3.6. Context Window và quản lý lịch sử hội thoại	10
2.4. RAG, embedding và Graph RAG.....	11

2.4.1. Retrieval-Augmented Generation.....	11
2.4.2. Embedding và cosine similarity	12
2.4.3. Truy hồi hybrid và Reciprocal Rank Fusion	13
2.4.4. Knowledge Graph và Graph RAG	13
2.5. Công nghệ nền của hệ thống	15
2.5.1. FastAPI và kiến trúc backend monolith	15
2.5.2. PostgreSQL, SQLAlchemy và Redis	15
2.5.3. Flutter, Riverpod và GoRouter	15
2.5.4. Neo4j và Cypher.....	15
2.6. Bảo mật và thanh toán	16
2.6.1. Xác thực và phân quyền	16
2.6.2. Thanh toán Stripe	16
2.6.3. Phạm vi prototype VNPAY và Docker Compose	16
2.7. Streaming phản hồi AI và giao tiếp thời gian thực.....	16
2.7.1. Server-Sent Events (SSE) trong streaming AI	16
2.7.2. WebSocket cho chat và thông báo realtime	17
2.7.3. Thiết kế xử lý bất đồng bộ trong backend AI.....	17
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	18
3.1. Mô tả bài toán.....	18
3.1.1. Các tác nhân của hệ thống.....	19
3.1.2. Yêu cầu chức năng	20
3.1.3. Yêu cầu phi chức năng	21
3.2. Thiết kế cơ sở dữ liệu	21
3.2.1. Các thực thể và thuộc tính	21
3.2.2. Xây dựng mô hình thực thể liên kết	29
3.2.3. Chuyển từ mô hình thực thể liên kết sang mô hình quan hệ.....	30
3.2.4. Chuẩn hóa	32
3.2.5. Thiết kế Knowledge Graph Neo4j.....	34
3.3.1. Danh sách tác nhân và ca sử dụng.....	35
3.3.2. Đặc tả các ca sử dụng quan trọng.....	40
3.3.3. Quan hệ use-case và ràng buộc nghiệp vụ	44
3.4. Thiết kế kiến trúc hệ thống.....	45
3.4.1. Kiến trúc monolith phân tầng.....	45

3.4.3. Phân bổ dữ liệu và trạng thái	46
3.5. Phân tích và thiết kế hệ thống AI Agent, Tool Calling và Graph RAG	47
3.5.1. Phân tích bài toán hội thoại và ý định người dùng	47
3.5.2. Kiến trúc nội bộ và ranh giới trách nhiệm	48
3.5.3. Thiết kế prompt, memory và ngữ cảnh tin cậy	48
3.5.4. Thiết kế Tool Registry và schema tham số	50
3.5.5. Luồng hoàn chỉnh của một Tool Calling	51
3.5.6. Cơ chế LangChain AgentExecutor và phân biệt Function/Tool Calling	51
3.5.7. Kỹ thuật truy hồi Graph RAG	52
3.5.8. Embedding, cosine và đồng bộ khởi động	53
3.5.9. Fallback, bảo mật và khả năng quan sát	54
3.5.10. Tiêu chí đánh giá hệ thống AI Agent	55
3.6. Thiết kế API và bảo mật	56
3.6.1. Các nhóm API chính	56
3.6.2. Xác thực, phân quyền và xử lý lỗi	56
3.7. Đánh giá thiết kế	57
3.7.1. Ưu điểm	57
3.7.2. Hạn chế và hướng hoàn thiện	57
CHƯƠNG 4: GIAO DIỆN CHƯƠNG TRÌNH	58
4.1. Nguyên tắc thiết kế giao diện	58
4.1.1. Đối tượng sử dụng và điều hướng theo vai trò	58
4.1.2. Tính nhất quán và phản hồi trạng thái	58
4.1.3. State management và kết nối backend	58
4.2. Giao diện khách hàng	58
4.2.1. Đăng nhập và màn hình chính	58
4.2.2. Giao diện Chat AI	61
4.2.3. Đặt sân và lịch sử booking	62
4.2.4. Menu, giỏ hàng và thanh toán	62
4.2.5. Gọi nhân viên và chat realtime	64
4.3. Giao diện nhân viên, quản trị và đối tác	65
4.3.1. Giao diện nhân viên	65
4.3.2. Giao diện quản trị viên	67
4.3.3. Giao diện đối tác F&B	69

4.4. Môi trường chạy và cấu hình.....	70
4.4.1. Thành phần môi trường phát triển.....	70
4.4.2. Cấu hình khởi động AI và Knowledge Graph.....	70
4.4.3. Phạm vi Docker Compose	70
4.5. Kết quả thực nghiệm và kiểm thử	71
4.5.1. Kết quả chức năng	71
4.5.2. Kết quả Graph RAG	71
4.5.3. Kết quả kiểm thử phần mềm	71
4.5.4. Giới hạn thực nghiệm	71
KẾT LUẬN VÀ KIẾN NGHỊ	73
TÀI LIỆU THAM KHẢO	74

This image shows a full page of a handwriting practice worksheet. It consists of multiple horizontal rows, each defined by two parallel dotted lines. The rows are evenly spaced and extend across the entire width of the page, providing a guide for letter height and placement. There is no text or other markings on the page.

GIÁO VIÊN HƯỚNG DẪN

i

LỜI MỞ ĐẦU

Trong bối cảnh công nghệ trí tuệ nhân tạo (AI) đang phát triển mạnh mẽ, việc ứng dụng AI vào quản lý và phục vụ khách hàng trong lĩnh vực thể thao giải trí ngày càng trở nên cần thiết. Đồ án này tập trung vào việc xây dựng hệ thống chatbot AI cho quán thể thao (bida, pickleball, cầu lông) tích hợp Knowledge Graph và Tool Calling.

Lý do chọn đề tài: Các quán thể thao hiện nay vẫn chủ yếu quản lý theo cách thủ công, thiếu hệ thống tự động hóa để hỗ trợ khách hàng tra cứu luật chơi, kỹ thuật, đặt sân và gọi đồ uống. Việc xây dựng chatbot AI giúp giải quyết các vấn đề trên, nâng cao trải nghiệm khách hàng và tối ưu hóa quy trình phục vụ. Bên cạnh đó, nhu cầu thanh toán trực tuyến ngày càng cao. Trong phạm vi đồ án, Stripe được tích hợp để thử nghiệm thanh toán; VNPay được nghiên cứu ở mức prototype nhằm khảo sát phương án tích hợp cổng thanh toán trong nước.

Mục tiêu nghiên cứu: Xây dựng hệ thống chatbot AI có khả năng (1) trả lời câu hỏi về luật chơi, kỹ thuật thể thao thông qua Knowledge Graph; (2) hỗ trợ đặt sân theo thời gian thực; (3) đặt đồ uống từ menu; (4) gọi nhân viên hỗ trợ; (5) tích hợp thanh toán Stripe và xây dựng prototype nghiên cứu VNPay; (6) cung cấp ứng dụng di động đa nền tảng.

Phương pháp nghiên cứu: Đồ án sử dụng phương pháp kết hợp giữa phát triển phần mềm và ứng dụng các công nghệ AI hiện đại (LangChain, LLM, Knowledge Graph). Dữ liệu tri thức thể thao được thu thập từ các nguồn chính thức (WPA, USAPA, BWF) và xây dựng thành Knowledge Graph trong Neo4j. Backend FastAPI được xây dựng theo kiến trúc monolith, tập trung các API và nghiệp vụ trong một ứng dụng.

Em xin cảm ơn thầy Lưu Toàn Định đã hướng dẫn em trong quá trình làm đồ án.

Do kinh nghiệm hạn chế, báo cáo chắc chắn còn thiếu sót. Em mong thầy cô và hội đồng góp ý để em cải thiện.

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Giải thích
AI	Artificial Intelligence - Trí tuệ nhân tạo
API	Application Programming Interface - Giao diện lập trình ứng dụng
BWF	Badminton World Federation - Liên đoàn Cầu lông Thế giới
CRUD	Create, Read, Update, Delete - Tạo, Đọc, Sửa, Xóa
CSS	Cascading Style Sheets
DIO	Dart HTTP client library
ERD	Entity-Relationship Diagram - Sơ đồ thực thể quan hệ
FK	Foreign Key - Khóa ngoại
gRPC	Google Remote Procedure Call
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment - Môi trường phát triển tích hợp
JSON	JavaScript Object Notation
JWT	JSON Web Token - Mã xác thực người dùng
KG	Knowledge Graph - Đồ thị tri thức
LLM	Large Language Model - Mô hình ngôn ngữ lớn
ORM	Object-Relational Mapping - Ánh xạ đối tượng-quan hệ
PK	Primary Key - Khóa chính
RAG	Retrieval Augmented Generation - Truy xuất tăng cường
REST	Representational State Transfer
RTSP	Real-Time Streaming Protocol
SDK	Software Development Kit - Bộ phát triển phần mềm
SQL	Structured Query Language - Ngôn ngữ truy vấn có cấu trúc
SQLAlchemy	Python ORM library cho PostgreSQL
UUID	Universally Unique Identifier - Định danh duy nhất toàn cầu
USAPA	USA Pickleball Association - Hiệp hội Pickleball Hoa Kỳ
VND	Vietnamese Dong - Đồng Việt Nam
VNPay	Cổng thanh toán trực tuyến Việt Nam
WebSocket	Giao thức truyền thông hai chiều thời gian thực
WPA	World Pool-Billiard Association - Hiệp hội Bida Thế giới

DANH MỤC BẢNG

Bảng 1.1. Công việc cần làm	2
Bảng 1.2. Phạm vi nghiên cứu	4
Bảng 2.1. Các giao thức dùng trong API	5
Bảng 2.2. Phân biệt chatbot LLM và AI Agent	8
Bảng 2.3. Các giai đoạn xử lý yêu cầu trong hệ thống Agent	9
Bảng 2.4. Các thành phần xử lý tri thức trong Graph RAG	15
Bảng 3.1. Các tác nhân của hệ thống	19
Bảng 3.2. Yêu cầu chức năng	20
Bảng 3.3. Yêu cầu phi chức năng	21
Bảng 3.4. Users (Người dùng): lưu danh tính, xác thực và vai trò	21
Bảng 3.5. Businesses (Đơn vị kinh doanh): nhóm các venue cùng chủ sở hữu	22
Bảng 3.6. Venues (Cơ sở): lưu địa điểm vận hành	22
Bảng 3.7. VenueAreas (Khu vực): phân chia khu vực trong venue	22
Bảng 3.8. ServiceResources (Tài nguyên phục vụ): lưu sân, bàn hoặc tài nguyên có thể đặt	23
Bảng 3.9. StaffAssignments (Phân công nhân viên): giới hạn phạm vi vận hành của staff	23
Bảng 3.10. Bookings (Đặt sân/bàn): lưu giao dịch đặt tài nguyên theo thời gian	24
Bảng 3.11. MenuItem (Món/dịch vụ): lưu menu riêng của từng venue.....	24
Bảng 3.12. Orders (Đơn tại venue): lưu đơn món/dịch vụ của khách	25
Bảng 3.13. OrderItems (Chi tiết đơn): lưu snapshot món và giá tại thời điểm đặt.....	25
Bảng 3.14. PaymentTransactions (Giao dịch thanh toán): lưu kết quả giao dịch công thanh toán	26
Bảng 3.15. StaffRequests (Yêu cầu nhân viên): lưu yêu cầu hỗ trợ và vòng đời xử lý.....	26
Bảng 3.16. Notifications (Thông báo): lưu sự kiện gửi đến người dùng theo vai trò.....	27
Bảng 3.17. Cameras (Camera giám sát): lưu cấu hình camera theo venue/resource....	27
Bảng 3.18. PartnerStores (Cửa hàng đối tác): lưu thông tin cửa hàng F&B	28
Bảng 3.19. PartnerMenuItem (Menu đối tác): lưu sản phẩm của cửa hàng đối tác	28
Bảng 3.20. PartnerOrders (Đơn đối tác): lưu đơn giao từ partner store	29
Bảng 3.21. PartnerOrderItems (Chi tiết đơn đối tác): lưu snapshot sản phẩm trong đơn	29

Bảng 3.22. Chuẩn hoá	33
Bảng 3.23. Các nhãn node trong Knowledge Graph Neo4j.....	34
Bảng 3.24. Các loại quan hệ trong Knowledge Graph Neo4j.....	34
Bảng 3.25. Bảng đặc tả UC05. Chat với AI Agent	40
Bảng 3.26. Bảng đặc tả UC06. Tra cứu tri thức ba môn.....	40
Bảng 3.27. Bảng đặc tả UC08. Tạo booking	41
Bảng 3.28. Bảng đặc tả UC12. Tạo và theo dõi order	41
Bảng 3.29. Bảng đặc tả UC13. Gọi nhân viên/yêu cầu hỗ trợ	42
Bảng 3.30. Bảng đặc tả UC16. Thanh toán qua Stripe	42
Bảng 3.31. Bảng đặc tả UC20. Xử lý staff request.....	43
Bảng 3.32. Bảng đặc tả UC25. Quản lý booking.....	43
Bảng 3.33 Bảng đặc tả UC34. Quản lý vòng đời partner order.....	44
Bảng 3.34. Các tầng kiến trúc của hệ thống.....	45
Bảng 3.35. Phân bố dữ liệu và trạng thái	46
Bảng 3.36. Các nhóm ý định và hướng xử lý của AI Agent.....	47
Bảng 3.37. Kiến trúc nội bộ và ranh giới trách nhiệm	48
Bảng 3.38. Bảng thiết kế prompt, memory và ngữ cảnh tin cậy.....	49
Bảng 3.39. Bảng thiết kế Tool Registry và schema tham số	50
Bảng 3.40. Các nguyên tắc thiết kế tool cho AI Agent.....	50
Bảng 3.41. Chính sách điều phối và xử lý Tool Calling.....	52
Bảng 3.42. Các tiêu chí xếp hạng kết quả truy xuất trong Graph RAG	53
Bảng 3.43. Các rủi ro kỹ thuật và cơ chế xử lý.....	54
Bảng 3.44. Các chỉ số đánh giá hệ thống AI Agent	55
Bảng 3.45. Các nhóm API chính của hệ thống	56
Bảng 4.1. Môi trường triển khai hệ thống.....	70
Bảng 4.2. Kết quả chức năng	71

DANH MỤC HÌNH ẢNH

Hình 3.1. Sơ đồ mô hình thực thể liên kết	30
Hình 3.2. Sơ đồ cấu trúc Knowledge Graph	35
Hình 3.3. Sơ đồ Use-case tổng quát (Customer)	37
Hình 3.4. Sơ đồ Use-case tổng quát (Staff)	38
Hình 3.5. Sơ đồ Use-case tổng quát (Admin/Manager)	39
Hình 3.6. Sơ đồ kiến trúc tổng thể	46
Hình 3.7. LLM Tool Calling sequence diagram	51
Hình 4.1. Màn hình đăng nhập	59
Hình 4.2. Màn hình chính của khách hàng	60
Hình 4.3. Màn hình Chat AI và thẻ kết quả tool	61
Hình 4.4. Màn hình đặt sân	62
Hình 4.5. Màn hình menu và giỏ hàng	63
Hình 4.6. Màn hình thanh toán Stripe	64
Hình 4.7. Gọi nhân viên và theo dõi yêu cầu	65
Hình 4.8. Cụm màn hình cho nhân viên (staff)	66
Hình 4.9. Cụm màn hình dashboard và quản trị (admin/quản lý)	68
Hình 4.10. Cụm màn hình dashboard và quản trị (partner/quản đối tác)	69

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Lý do chọn đề tài

Bida, pickleball và cầu lông là các môn thể thao giải trí có nhu cầu sử dụng sân, bàn, thiết bị và dịch vụ đi kèm theo thời gian thực. Tại nhiều cơ sở, việc đặt sân, gọi món, yêu cầu nhân viên và tra cứu thông tin vẫn được thực hiện qua điện thoại, tin nhắn hoặc trao đổi trực tiếp. Quy trình này phụ thuộc nhiều vào nhân viên, khó duy trì lịch sử thống nhất và dễ phát sinh sai sót khi lượng khách tăng.

Chatbot thông thường có thể trả lời câu hỏi nhưng không tự tạo booking hay đơn hàng đáng tin cậy. Mô hình ngôn ngữ lớn cũng có nguy cơ sinh thông tin không có trong dữ liệu. Đề tài lựa chọn hướng AI Agent kết hợp Tool Calling: LLM chịu trách nhiệm hiểu yêu cầu và sinh lời gọi có cấu trúc; mã nguồn Python, service và repository mới là thành phần thực thi nghiệp vụ thật.

Đối với câu hỏi về luật và kỹ thuật thể thao, đề tài không chỉ dựa vào kiến thức tham số của LLM mà xây dựng Knowledge Graph từ các nguồn WPA, USA Pickleball và BWF. Graph RAG kết hợp full-text, keyword, vector search và quan hệ Neo4j để đưa ngữ cảnh truy xuất được vào câu trả lời. Cách tiếp cận này phù hợp với mục tiêu nghiên cứu về grounding và giảm hallucination.

Đề tài đồng thời có giá trị thực hành vì bao quát ứng dụng di động Flutter, backend FastAPI, PostgreSQL, Redis, Neo4j, WebSocket, thanh toán Stripe và quản trị đa vai trò. Phạm vi lớn phù hợp với đề án tốt nghiệp và cho phép đánh giá một hệ thống AI tích hợp nghiệp vụ thay vì chỉ đánh giá mô hình chat.

1.2. Mục tiêu, nội dung và phương pháp nghiên cứu đề tài

1.2.1. Mục tiêu nghiên cứu

Mục tiêu tổng quát là xây dựng hệ thống chatbot AI hỗ trợ vận hành quán thể thao và cung cấp giao diện hội thoại cho các chức năng nghiệp vụ. Hệ thống phải phân biệt rõ giữa khả năng suy luận ngôn ngữ của LLM với khả năng thực thi của tool và service.

- Xây dựng ứng dụng Flutter đa vai trò cho khách hàng, nhân viên, quản trị viên và đối tác.
- Xây dựng backend FastAPI monolith, phân tầng API - service - repository - model.

- Tích hợp LangChain AgentExecutor và tập tool có schema để thực hiện nghiệp vụ có kiểm soát.
- Xây dựng Knowledge Graph cho ba môn và đưa truy hồi hybrid vào tool query_knowledge.
- Lưu dữ liệu nghiệp vụ trong PostgreSQL; dùng Redis cho session, cache và realtime.
- Kiểm thử luồng Stripe; mô tả chính xác VNPay chỉ là prototype nghiên cứu.
- Đánh giá kết quả, giới hạn retrieval và hướng phát triển sau đồ án.

1.2.2. Nội dung nghiên cứu

Nội dung nghiên cứu được chia thành các nhóm liên kết với nhau. Nhóm thứ nhất khảo sát nghiệp vụ quán thể thao, actor, use-case và quy tắc dữ liệu. Nhóm thứ hai nghiên cứu nền tảng AI gồm LLM, Agent, Tool Calling, embedding, RAG, Graph RAG và cơ chế xếp hạng. Nhóm thứ ba thiết kế và hiện thực phần mềm. Nhóm cuối cùng thực nghiệm, kiểm thử và đánh giá.

Nhóm nội dung	Công việc chính	Kết quả
Khảo sát nghiệp vụ	Phân tích đặt sân, order, hỗ trợ nhân viên, thanh toán và quản trị	Yêu cầu chức năng, phi chức năng và use-case
Nghiên cứu AI	LLM, Agent, Tool Calling, LangChain, RAG, embedding và Neo4j	Mô hình xử lý hội thoại có tool và Graph RAG
Thiết kế hệ thống	Kiến trúc monolith phân tầng, dữ liệu, API, bảo mật và realtime	Thiết kế thành phần và hợp đồng dữ liệu
Hiện thực	FastAPI, Flutter, PostgreSQL, Redis, Neo4j và Stripe	Ứng dụng chạy được trong môi trường phát triển
Đánh giá	Unit test, integration smoke test và đánh giá giới hạn	Kết quả thực nghiệm và kiến nghị

Bảng 1.1. Công việc cần làm

1.2.3. Phương pháp nghiên cứu

Đề tài sử dụng phương pháp nghiên cứu ứng dụng, kết hợp tổng hợp tài liệu, phân tích hệ thống, thiết kế phần mềm, thực nghiệm và đánh giá. Các khái niệm AI được đối chiếu với cách hiện thực trong mã nguồn để tránh mô tả lý thuyết không phản ánh hệ thống.

- Thu thập yêu cầu từ bối cảnh vận hành quán thể thao và xác định actor.
- Nghiên cứu tài liệu chính thức của FastAPI, Flutter, LangChain, Neo4j, Ollama và Stripe.
- Thu thập dữ liệu luật, kỹ thuật từ WPA, USA Pickleball và BWF; chuẩn hóa thành entity và relationship.
- Thiết kế lập theo từng module; ưu tiên luồng chat, booking, order, staff request và knowledge retrieval.
- Kiểm thử từng tool/service độc lập, sau đó kiểm tra tích hợp với database và ứng dụng.
- Đánh giá định tính kết quả retrieval; ghi nhận trung thực các thành phần chưa đủ điều kiện kiểm thử.

1.3. Đối tượng và phạm vi nghiên cứu

1.3.1. Đối tượng nghiên cứu

Đối tượng nghiên cứu chính là cơ chế xây dựng một AI Agent có khả năng chuyển yêu cầu tự nhiên thành hành động nghiệp vụ an toàn. Các đối tượng kỹ thuật liên quan gồm LLM, schema tool, AgentExecutor, context hội thoại, service/repository, Knowledge Graph và retrieval hybrid.

- Người dùng hệ thống: Customer, Staff, Admin và Partner.
- Nghiệp vụ: đặt sân, đặt món, xem lịch, gọi nhân viên, thanh toán và quản trị.
- Tri thức: luật, kỹ thuật, dụng cụ và khái niệm của bida, pickleball, cầu lông.
- Công nghệ: Flutter, FastAPI, LangChain, PostgreSQL, Redis, Neo4j, Ollama và Stripe.

1.3.2. Phạm vi nghiên cứu

Phạm vi chức năng tập trung vào một nền tảng quản lý quán thể thao đa venue. Backend hiện thực dưới dạng monolith: các router, service, repository và AI Agent nằm

trong cùng ứng dụng FastAPI. PostgreSQL, Redis và Neo4j là các hạ tầng dữ liệu bên ngoài, không làm cho backend trở thành microservices.

Phạm vi	Bao gồm	Không bao gồm
Môn thể thao	Bida, pickleball, cầu lông	Không bao phủ toàn bộ môn thể thao
AI	Tool Calling, memory, intent routing, Graph RAG	LLM không trực tiếp chạy SQL/Cypher hoặc business logic
Thanh toán	Stripe test mode; mã prototype VNPAY	VNPAY chưa hoạt động sandbox do thiếu merchant credentials
Triển khai	Môi trường phát triển cục bộ và Neo4j Aura	Docker Compose không triển khai toàn hệ thống
Đánh giá	Unit test, smoke test và đánh giá định tính	Chưa có benchmark retrieval quy mô lớn

Bảng 1.2. Phạm vi nghiên cứu

1.4. Kết cấu báo cáo

Báo cáo gồm bốn chương. Chương 1 trình bày tổng quan đề tài. Chương 2 tổng hợp cơ sở lý thuyết, trong đó trọng tâm là AI Agent, LangChain, Tool Calling và Graph RAG. Chương 3 phân tích và thiết kế hệ thống, mô tả cách các khái niệm được ánh xạ vào kiến trúc và mã nguồn. Chương 4 trình bày giao diện chương trình, môi trường chạy và kết quả đánh giá. Sau đó là kết luận, kiến nghị, tài liệu tham khảo và phụ lục.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. API và giao tiếp giữa các thành phần

2.1.1. API (Application Programming Interface)

API là hợp đồng cho phép các thành phần phần mềm trao đổi dữ liệu mà không cần biết chi tiết cài đặt bên trong của nhau. Hợp đồng API mô tả endpoint hoặc tên hàm, kiểu tham số, dữ liệu trả về, lỗi có thể xảy ra và quy tắc xác thực. Trong đề tài, Flutter không truy cập database trực tiếp mà gọi API FastAPI; AI Agent cũng không truy cập service tùy ý mà thông qua tool schema.^[1]

2.1.2. Các giao thức thường dùng trong API

Cơ chế	Đặc điểm	Ứng dụng trong đề tài
HTTP/REST	Request-response, tài nguyên và status code	API nghiệp vụ FastAPI
WebSocket	Kết nối hai chiều, độ trễ thấp	Chat nhân viên và thông báo realtime
SSE	Server phát luồng dữ liệu một chiều qua HTTP	Streaming phản hồi AI
gRPC	RPC nhị phân, hợp đồng protobuf	Prototype Java VNPAY

Bảng 2.1. Các giao thức dùng trong API

2.1.3. Ưu điểm của việc sử dụng API

- Tách giao diện Flutter khỏi cách lưu và xử lý dữ liệu ở backend.
- Chuẩn hóa xác thực, kiểm tra dữ liệu và phản hồi lỗi.
- Cho phép kiểm thử độc lập từng endpoint hoặc service.
- Hạn chế quyền của AI Agent thông qua danh sách tool được đăng ký.
- Hỗ trợ mở rộng client web hoặc tích hợp bên thứ ba trong tương lai.

2.2. Mô hình ngôn ngữ lớn và AI Agent

2.2.1. Mô hình ngôn ngữ lớn (LLM)

LLM là mô hình học sâu được huấn luyện trên lượng lớn văn bản để dự đoán token tiếp theo. Nhờ đó, mô hình có thể hiểu chỉ dẫn, tổng hợp thông tin và sinh phản hồi tự

nhiên. Tuy nhiên, LLM không mặc nhiên có quyền truy cập dữ liệu mới nhất, không bảo đảm tính đúng của phép tính nghiệp vụ và có thể hallucinate khi thiếu ngữ cảnh. Trong hệ thống, LLM được dùng cho hiểu ý định, suy luận tham số và diễn đạt kết quả. Dữ liệu động như sân trống, giá, booking hay đơn hàng phải lấy từ database thông qua tool. Quy tắc này tạo ranh giới rõ giữa sinh ngôn ngữ và thực thi nghiệp vụ.

a) Kiến trúc Transformer và cơ chế Attention

Transformer là kiến trúc mạng nơ-ron sâu được giới thiệu trong công trình "Attention is All You Need" (Vaswani et al., 2017). Trước Transformer, các mô hình xử lý chuỗi như RNN và LSTM phải xử lý tuần tự từng token, dẫn đến khó học phụ thuộc xa và chậm khi huấn luyện song song. Transformer giải quyết hạn chế này bằng cách xử lý toàn bộ chuỗi đầu vào đồng thời thông qua cơ chế self-attention.^[20]

Cơ chế self-attention cho phép mỗi token trong chuỗi "chú ý" đến tất cả các token khác với mức độ khác nhau. Cụ thể, mỗi token được chiếu thành ba vector: Query (Q), Key (K) và Value (V). Điểm attention giữa token i và token j được tính theo công thức $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$, trong đó d_k là chiều của vector Key dùng để chuẩn hóa. Điểm số này xác định mức độ ảnh hưởng của token j lên biểu diễn của token i .^[3]

Multi-head attention mở rộng cơ chế trên bằng cách chạy nhiều head song song, mỗi head học một loại phụ thuộc ngữ nghĩa khác nhau. Kết quả từ các head được nối lại và chiếu qua một lớp tuyến tính. Nhờ đó, mô hình có thể học cả quan hệ cú pháp, ngữ nghĩa và tham chiếu trong cùng một bước.^[20]

Positional encoding được thêm vào embedding đầu vào để cung cấp thông tin về vị trí token trong chuỗi, vì self-attention bản thân không nhạy cảm với thứ tự. Các LLM hiện đại thường dùng kiến trúc decoder-only (như GPT), trong đó mỗi token chỉ attend đến các token phía trước để phù hợp với bài toán sinh văn bản tự hồi quy (autoregressive).^[2]

b) Quá trình huấn luyện và tinh chỉnh LLM

Quá trình xây dựng LLM hiện đại trải qua ba giai đoạn chính. Giai đoạn pre-training huấn luyện mô hình trên corpus văn bản rất lớn với mục tiêu dự đoán token tiếp theo (causal language modeling). Giai đoạn này tốn kém nhất về tài nguyên tính toán nhưng cho phép mô hình học biểu diễn ngôn ngữ tổng quát. Giai đoạn instruction tuning

(supervised fine-tuning) sau đó huấn luyện mô hình trên các cặp instruction-response có nhãn để mô hình hiểu và thực thi chỉ dẫn.^[20]

Giai đoạn Reinforcement Learning from Human Feedback (RLHF) căn chỉnh hành vi mô hình với kỳ vọng của người dùng. Người đánh giá xếp hạng các phản hồi do mô hình sinh ra; xếp hạng này được dùng để huấn luyện một reward model; sau đó PPO (Proximal Policy Optimization) tối ưu mô hình chính để tối đa hóa reward. Nhờ RLHF, mô hình không chỉ trả lời đúng ngữ pháp mà còn có xu hướng hữu ích, trung thực và không gây hại.^[20]

Hạn chế cần nhận thức rõ: LLM có thể "hallucinate" — sinh ra thông tin nghe có vẻ hợp lý nhưng không chính xác — đặc biệt khi bị hỏi về dữ liệu cụ thể, số liệu hoặc sự kiện ngoài tập huấn luyện. Đây là lý do hệ thống trong đề tài tách biệt rõ vai trò của LLM (sinh ngôn ngữ, suy luận ý định) và role của tool/service (thực thi nghiệp vụ, lấy dữ liệu thật từ database).

2.2.2. Khái niệm AI Agent

AI Agent là hệ thống sử dụng mô hình để lựa chọn hành động, quan sát kết quả và tiếp tục suy luận cho đến khi hoàn thành mục tiêu hoặc đạt giới hạn. Một agent cơ bản gồm model, prompt, memory/context, tool registry và vòng lặp điều phối. Agent không đồng nghĩa với LLM; LLM chỉ là thành phần ra quyết định trong một kiến trúc lớn hơn.^[3]

a) Kiến trúc nội bộ của AI Agent và vòng lặp ReAct

Vòng lặp ReAct (Reason + Act) là một pattern phổ biến để xây dựng AI Agent. Trong mỗi bước, agent quan sát trạng thái hiện tại, suy luận để chọn hành động, thực thi hành động và nhận observation từ môi trường. Vòng lặp tiếp tục cho đến khi agent sinh ra câu trả lời cuối hoặc đạt giới hạn số bước. Cách tiếp cận này tạo ra agent có thể xử lý bài toán nhiều bước phức tạp mà một lần gọi LLM không thể giải quyết.^[3]

Agent scratchpad là vùng lưu trữ trung gian chứa lịch sử suy luận, các tool call đã thực hiện và kết quả nhận được. Toàn bộ scratchpad được đưa vào prompt cho LLM ở mỗi bước, giúp model duy trì ngữ cảnh qua nhiều vòng lặp.

b) Phân loại bộ nhớ trong AI Agent

Bộ nhớ của AI Agent được phân thành bốn loại theo phạm vi và thời gian lưu trữ. In-context memory là thông tin trong context window của LLM trong một phiên hội thoại, bao gồm chat history và agent scratchpad; loại này nhanh nhất nhưng bị giới hạn

bởi kích thước context window. External memory là cơ sở dữ liệu bên ngoài (vector store, graph database, SQL database) mà agent truy cập qua tool; loại này không giới hạn kích thước nhưng cần bước retrieval.^[3]

In-weights memory là kiến thức đã được mã hóa vào tham số của mô hình trong quá trình pre-training; loại này không thay đổi sau khi deploy và không phản ánh dữ liệu mới. Cache memory là kết quả đã tính trước (ví dụ vector embedding của IntentRouter lưu ở Redis) để tránh tính lại. Trong đề tài, hệ thống kết hợp in-context memory (chat history), external memory (Neo4j, PostgreSQL) và cache memory (Redis) để đảm bảo agent có đủ ngữ cảnh mà không tốn tài nguyên thừa.

2.2.3. Phân biệt chatbot LLM và AI Agent

Tiêu chí	Chatbot LLM	AI Agent trong đề tài
Luồng	Message → LLM → Text	Message → LLM → Tool → Observation → LLM
Dữ liệu động	Không bảo đảm cập nhật	Lấy từ PostgreSQL/Neo4j qua tool
Hành động	Chỉ sinh văn bản	Có thể tạo booking/order qua service
Kiểm soát	Phụ thuộc prompt	Giới hạn tool, schema, validation và số vòng lặp
Khả năng kiểm thử	Khó tách logic	Tool và service có thể unit test độc lập

Bảng 2.2. Phân biệt chatbot LLM và AI Agent

2.3. Tool Calling và Function Calling

2.3.1. Khái niệm và cách gọi thuật ngữ

Function Calling là khả năng của model sinh tên hàm và arguments theo JSON schema. Tool Calling là khái niệm rộng hơn: tool có thể là function Python, API, truy vấn dữ liệu hoặc một thao tác bên ngoài. Trong đồ án, mỗi LangChain tool được hiện thực bằng function Python, vì vậy có thể nói model sinh một function call, còn agent thực hiện một tool call.^[3]

Điểm quan trọng là LLM không tự chạy function. Model chỉ sinh dữ liệu điều khiển, ví dụ tên `book\_court` và các tham số thời gian. AgentExecutor nhận dữ liệu này, kiểm tra tool registry rồi mới gọi coroutine Python tương ứng.

2.3.2. Schema và cơ chế sinh tham số

Schema mô tả tên tool, mục đích, các trường đầu vào, kiểu dữ liệu và trường bắt buộc. LangChain tạo schema từ type hint, Pydantic model và docstring. Khi người dùng nói “mai 8 giờ đặt bàn bida trong hai tiếng”, LLM dựa vào message, ngày giờ hiện tại, venue context và schema để sinh `court\_type`, `court\_number`, `start\_time`, `end\_time` và `notes`. Đây chính là sinh tham số.

Giai đoạn	Thành phần thực hiện	Kết quả
Hiểu câu	LLM	Nhận diện người dùng muốn đặt sân
Chọn tool	LLM	Chọn book_court
Sinh tham số	LLM	JSON theo schema của book_court
Kiểm tra	LangChain/Pydantic/tool	Kiểm dữ liệu và trường bắt buộc
Nghiệp vụ	Service/repository	Kiểm tra trùng lịch và ghi PostgreSQL
Diễn đạt	LLM	Câu trả lời tự nhiên từ kết quả thật

Bảng 2.3. Các giai đoạn xử lý yêu cầu trong hệ thống Agent

2.3.3. Luồng hoàn chỉnh của một tool call

- Backend nhận message, xác thực người dùng và nạp chat history/context.
- AgentExecutor gửi prompt, history và danh sách tool schema cho LLM.
- LLM trả lời trực tiếp hoặc sinh tool call gồm tên tool và arguments.
- AgentExecutor ánh xạ tên tool với function đã đăng ký và gọi function.
- Tool gọi service/repository, truy vấn database hoặc thực hiện nghiệp vụ.
- Kết quả tool được ghi vào agent scratchpad dưới dạng observation.
- LLM nhận observation và quyết định gọi thêm tool hoặc tạo câu trả lời cuối.
- Backend trả text, tools_used và metadata có cấu trúc cho Flutter.

2.3.4. LangChain và AgentExecutor

LangChain là framework cung cấp lớp trừu tượng cho model, prompt, message, tool và agent. `create_tool_calling_agent` xây dựng agent có khả năng sử dụng tool schema; `AgentExecutor` điều khiển vòng lặp hành động và observation. Trong đề tài, số vòng lặp được giới hạn, lỗi parse được xử lý và intermediate steps được giữ lại để xác định tool đã sử dụng.^[3]

2.3.5. Prompt Engineering và System Prompt

Prompt Engineering là kỹ thuật thiết kế đầu vào cho LLM để thu được đầu ra mong muốn một cách nhất quán. System prompt là phần chỉ dẫn được đặt trước toàn bộ hội thoại, xác định vai trò, hạn chế, định dạng đầu ra và ngữ cảnh nghiệp vụ của agent. Khác với user message, system prompt không hiển thị cho người dùng cuối và thường được backend kiểm soát.^[3]

Trong đề tài, system prompt cung cấp cho agent các thông tin: ngày giờ hiện tại, thông tin venue đang phục vụ, vai trò người dùng, danh sách tool có sẵn và quy tắc hành vi. Ví dụ, agent được chỉ dẫn không tự sinh user_id hay venue_id mà phải lấy từ context do backend cung cấp. Thiết kế này ngăn mô hình giả mạo danh tính hoặc truy cập dữ liệu ngoài phạm vi được phép.

Few-shot prompting là kỹ thuật đưa một số ví dụ (input, output) vào prompt để hướng dẫn mô hình theo định dạng hoặc hành vi mong muốn. Chain-of-thought prompting khuyến khích mô hình thể hiện các bước suy luận trung gian trước khi đưa ra kết luận, giúp cải thiện độ chính xác trên các bài toán lập luận nhiều bước. Trong bối cảnh agent, kỹ thuật này thể hiện qua việc mô hình sinh Thought trước khi chọn Action.^[3]

2.3.6. Context Window và quản lý lịch sử hội thoại

Context window là số lượng token tối đa mà LLM có thể xử lý trong một lần gọi, bao gồm cả system prompt, lịch sử hội thoại, agent scratchpad và phản hồi sinh ra. Các mô hình hiện đại có context window từ 32K đến hàng triệu token, nhưng chi phí tính toán tăng theo bình phương độ dài đầu vào do cơ chế attention.

Trong hệ thống, lịch sử hội thoại (chat history) được lưu ở Redis với TTL và chỉ giữ lại N message gần nhất trước khi đưa vào prompt. Kỹ thuật này cân bằng giữa duy

trì ngữ cảnh hội thoại và không vượt quá context window. Khi agent có nhiều vòng lặp tool call, intermediate steps cũng được giữ lại trong context của lần gọi đó để LLM có thể quan sát toàn bộ quá trình suy luận.^{[3][7]}

2.4. RAG, embedding và Graph RAG

2.4.1. Retrieval-Augmented Generation

RAG là kỹ thuật truy xuất tài liệu liên quan trước khi yêu cầu LLM tạo câu trả lời. Thay vì chỉ dựa vào tham số đã học, model nhận thêm context từ nguồn dữ liệu của hệ thống. Quy trình RAG gồm lập chỉ mục, truy xuất, xây dựng context và sinh câu trả lời có grounding.^[13]

a) Pipeline RAG: từ lập chỉ mục đến sinh câu trả lời

Pipeline RAG đầy đủ gồm hai pha chính. Pha lập chỉ mục (offline) xử lý tài liệu nguồn qua các bước: thu thập và làm sạch dữ liệu, chia nhỏ thành chunks, sinh embedding cho mỗi chunk và lưu vào vector store kèm metadata. Pha truy xuất (runtime) nhận truy vấn từ người dùng, embed truy vấn bằng cùng model dùng khi lập chỉ mục, tìm kiếm top-k chunks gần nhất theo cosine similarity, xây dựng context prompt từ các chunk tìm được và gọi LLM để sinh câu trả lời có grounding.^[13]

Chiến lược chunking ảnh hưởng lớn đến chất lượng retrieval. Fixed-size chunking chia tài liệu theo số token cố định (thường 300–500 token) đơn giản nhưng có thể cắt đứt ý giữa câu. Semantic chunking chia theo ranh giới ngữ nghĩa tự nhiên như đoạn văn hoặc phần. Hierarchical chunking lưu cả chunk nhỏ (để match chính xác) lẫn chunk lớn (để cung cấp ngữ cảnh đầy đủ cho LLM). Nghiên cứu cho thấy adaptive chunking đạt độ chính xác cao hơn đáng kể so với fixed-size trong các tác vụ trả lời câu hỏi chuyên ngành.^[18]

Một pipeline RAG hiệu quả cũng cần bước post-retrieval. Re-ranking dùng cross-encoder model để tính điểm liên quan chính xác hơn cho top-k kết quả ban đầu, loại bỏ hiện tượng "lost in the middle" khi LLM bỏ qua thông tin quan trọng ở giữa danh sách context. Metadata filtering cho phép lọc kết quả theo thuộc tính như thời gian, loại tài liệu hoặc domain trước khi đưa vào LLM.^[13]

Pipeline RAG hoàn chỉnh bao gồm các bước nối tiếp nhau. Bước đầu tiên là thu thập và làm sạch dữ liệu: chuẩn hóa định dạng, loại bỏ nội dung nhiễu như header/footer thừa, đảm bảo corpus đầu vào có chất lượng tốt trước khi đưa vào lập chỉ mục. Tiếp theo

là chunking — chia tài liệu thành các đoạn nhỏ phù hợp với giới hạn context của embedding model, thường dùng RecursiveTextSplitter hoặc SemanticChunker của LangChain.

Sau chunking, mỗi đoạn được chuyển thành vector embedding và lưu vào cơ sở dữ liệu vector — trong đề tài là Neo4j vector index — kèm metadata nguồn gốc. Tại thời điểm truy vấn, hệ thống embed câu hỏi người dùng bằng cùng model rồi tìm top-k chunks gần nhất theo cosine similarity. Kết quả có thể qua bước re-ranking để loại bỏ chunk kém liên quan trước khi đưa vào context của LLM để sinh câu trả lời cuối cùng.

2.4.2. Embedding và cosine similarity

Embedding biểu diễn văn bản thành vector số. Các văn bản gần nghĩa được kỳ vọng có hướng vector gần nhau. Cosine similarity đo góc giữa hai vector theo công thức

$$\text{cosine similarity} = S_C(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}},$$

Giá trị cao cho biết hai vector có hướng gần nhau, nhưng chất lượng kết quả vẫn phụ thuộc model embedding.^[5]

Hệ thống dùng `nomic-embed-text`, vector 768 chiều và các task prefix `search\_document`, `search\_query`, `classification`. Embedding node được lưu trong Neo4j. Startup chỉ đồng bộ node thiếu hoặc stale; vector mẫu của IntentRouter được cache Redis để tránh gọi Ollama hàng loạt sau mỗi restart.

a) Các loại mô hình embedding và chỉ số HNSW

Mô hình embedding có thể phân theo kiến trúc và phạm vi sử dụng. Bi-encoder (như nomic-embed-text, text-embedding-3) encode query và document độc lập thành vector, cho phép tính toán offline và tìm kiếm nhanh qua vector index. Cross-encoder encode cả query lẫn document cùng lúc, cho độ chính xác cao hơn nhưng chậm hơn nên thường dùng ở giai đoạn re-ranking sau khi đã lọc top-k. Bi-encoder phù hợp với pha retrieval vì tốc độ; cross-encoder phù hợp với pha re-ranking vì độ chính xác.^[5]

HNSW (Hierarchical Navigable Small World) là cấu trúc chỉ mục xấp xỉ (Approximate Nearest Neighbor — ANN) cho phép tìm kiếm vector nhanh trên tập dữ liệu lớn. Thay vì so sánh tuyến tính tất cả vector, HNSW xây dựng một đồ thị phân cấp

kết nối các vector gần nhau. Tại thời điểm truy vấn, thuật toán bắt đầu từ điểm entry trên tầng cao nhất, di chuyển tham lam đến node gần nhất, sau đó xuống tầng thấp hơn cho đến khi đạt tầng đáy. Neo4j vector index dùng HNSW để đảm bảo tìm kiếm embedding trong thời gian logarithmic thay vì tuyến tính.^[4]

Chất lượng embedding phụ thuộc nhiều vào ngôn ngữ và domain của dữ liệu. Model nomic-embed-text được huấn luyện chủ yếu trên văn bản tiếng Anh, nên khi áp dụng cho tri thức thể thao tiếng Việt, chất lượng semantic search bị hạn chế — đây là điểm yếu được thừa nhận trong đề tài và là hướng cải thiện với các model đa ngôn ngữ như BGE-M3.

2.4.3. Truy hồi hybrid và Reciprocal Rank Fusion

Full-text search mạnh với từ khóa chính xác, còn vector search có thể tìm nội dung gần nghĩa. Hai nguồn có thang điểm khác nhau nên không cộng score thô trực tiếp. Reciprocal Rank Fusion hợp nhất theo thứ hạng với công thức $RRF(d) = \sum 1/(k + \text{rank}_i(d))$. Cách này giảm phụ thuộc vào scale của từng retriever và ưu tiên tài liệu xuất hiện ở nhiều danh sách.^[16]

2.4.4. Knowledge Graph và Graph RAG

Knowledge Graph biểu diễn tri thức bằng node và relationship. Neo4j phù hợp khi câu trả lời cần xem xét quan hệ như kỹ thuật thuộc môn nào, dụng cụ dùng cho kỹ thuật nào hoặc luật quy định khái niệm nào. Graph RAG trong đề tài truy xuất node seed bằng full-text/vector, sau đó mở rộng quan hệ một đến hai bước để bổ sung context.^[4]

a) So sánh Graph RAG và RAG truyền thống

RAG truyền thống dựa trên flat vector search: mỗi chunk là một đơn vị độc lập trong không gian vector, không có mối liên kết cấu trúc với các chunk khác. Khi câu hỏi yêu cầu tổng hợp thông tin từ nhiều thực thể có quan hệ với nhau (multi-hop reasoning), RAG thường thất bại vì không có cơ chế duyệt theo quan hệ. Graph RAG bổ sung lớp cấu trúc bằng cách biểu diễn tri thức dưới dạng đồ thị với node và relationship, cho phép agent không chỉ tìm node liên quan mà còn mở rộng theo đường đi trong đồ thị để thu thêm context.^[4]

Ví dụ minh họa: khi người dùng hỏi "Kỹ thuật phát bóng trong bóng đá bàn có luật gì không?", vector search có thể tìm được node "phát bóng" nhưng bỏ qua node "Luật thi đấu" nếu không đủ similarity. Graph traversal từ node "phát bóng" theo relationship

QUY_DINH sẽ dẫn trực tiếp đến node luật liên quan, cung cấp context chính xác hơn. Đây là ưu thế cốt lõi của Graph RAG: khai thác cấu trúc quan hệ để truy xuất thông tin theo ngữ nghĩa đồ thị, không chỉ dựa vào điểm similarity.^{[4][16]}

RAG truyền thống và Graph RAG khác nhau cơ bản ở cấu trúc dữ liệu và khả năng suy luận. RAG thường lưu chunks dạng flat trong vector store; retrieval chỉ dựa trên cosine similarity nên không có cơ chế nào để duyệt theo quan hệ giữa các thực thể. Graph RAG biểu diễn tri thức bằng node và relationship có thuộc tính, cho phép kết hợp full-text search, vector search và graph traversal trong cùng một pipeline.

Về multi-hop reasoning, RAG thường thất bại khi câu hỏi cần tổng hợp thông tin từ nhiều thực thể có liên hệ với nhau, vì không có liên kết cấu trúc giữa các chunk. Graph RAG giải quyết điều này bằng cách mở rộng 1–2 bước theo relationship từ node seed, thu thêm context liên quan mà vector similarity có thể bỏ sót. Đổi lại, Graph RAG yêu cầu đầu tư ban đầu để xây dựng ontology và cấu trúc hóa dữ liệu nguồn, phù hợp nhất khi tri thức có quan hệ rõ ràng như luật, kỹ thuật và dụng cụ trong hệ thống thể thao.

b) Xây dựng Knowledge Graph cho hệ thống thể thao

Để xây dựng Knowledge Graph, tri thức từ tài liệu về ba môn thể thao (bóng bàn/bida, pickleball, cầu lông) được cấu trúc hóa thành các thực thể và quan hệ. Mỗi node mang thuộc tính name, description và embedding vector. Relationship được gán nhãn ngữ nghĩa: THUOC (kỹ thuật thuộc môn nào), QUY_DINH (luật quy định khái niệm nào), DUNG_DE (dụng cụ dùng để làm gì), SU_DUNG (kỹ thuật sử dụng dụng cụ nào), LA_LOAI (phân loại cha-con), LIEN_QUAN (liên quan ngữ nghĩa tổng quát). Cấu trúc quan hệ này cho phép Cypher query duyệt đồ thị theo ngữ nghĩa thay vì chỉ theo text matching.^[4]

Pipeline đồng bộ embedding được thiết kế tăng dần: khi backend khởi động, hệ thống chỉ sinh embedding cho node chưa có hoặc có stale (node được cập nhật sau lần sync cuối). Kết quả embedding được lưu thẳng vào Neo4j làm thuộc tính của node, phục vụ cả full-text index và vector index. Thiết kế này tránh việc phải re-embed toàn bộ 418 node mỗi khi restart, giảm tải cho Ollama và giảm độ trễ khởi động.

Thành phần	Vai trò
Node	Rule, Technique, Equipment, Sport, Concept, GameType
Relationship	THUOC, QUY_DINH, DUNG_DE, SU_DUNG, LA_LOAI, LIEN QUAN
Full-text index	Tìm name/description theo từ khóa
Vector index	Tìm node theo cosine similarity
Graph traversal	Lấy quan hệ lân cận 1-2 bước
LLM	Tổng hợp context thành câu trả lời tự nhiên

Bảng 2.4. Các thành phần xử lý tri thức trong Graph RAG

2.5. Công nghệ nền của hệ thống

2.5.1. FastAPI và kiến trúc backend monolith

FastAPI là framework Python hỗ trợ async/await, Pydantic validation và tài liệu OpenAPI. Backend của đề tài là monolith phân tầng: các API, service, repository và agent chạy trong cùng tiến trình ứng dụng. Việc dùng nhiều database hoặc có prototype Java riêng không đủ để gọi toàn hệ thống là microservices.^[1]

2.5.2. PostgreSQL, SQLAlchemy và Redis

PostgreSQL lưu dữ liệu nghiệp vụ cần transaction và ràng buộc quan hệ. SQLAlchemy cung cấp ORM và truy vấn bất đồng bộ. Redis lưu chat session, cache kết quả, intent cache, presence và dữ liệu realtime có TTL. Sự phân công này giúp mỗi loại dữ liệu dùng đúng mô hình lưu trữ.^{[6][7][17]}

2.5.3. Flutter, Riverpod và GoRouter

Flutter xây dựng ứng dụng đa nền tảng từ một codebase Dart. Riverpod quản lý state theo provider, hỗ trợ dependency injection và async state. GoRouter quản lý route theo vai trò và trạng thái xác thực. Dio đảm nhiệm HTTP, còn WebSocket/SSE phục vụ dữ liệu thời gian thực.^{[2][18]}

2.5.4. Neo4j và Cypher

Neo4j là graph database theo mô hình property graph. Cypher mô tả pattern node-relationship trực quan và hỗ trợ index, full-text, vector search cùng traversal. Ưu thế của

Neo4j chỉ được khai thác khi hệ thống dùng quan hệ để mở rộng hoặc giải thích kết quả, không chỉ lưu document như collection.^[4]

2.6. Bảo mật và thanh toán

2.6.1. Xác thực và phân quyền

Hệ thống dùng JWT Bearer token, hash mật khẩu, Pydantic validation và kiểm tra role/venue. Context nhạy cảm như user_id không để LLM tự sinh mà được backend gán bằng ContextVar trước khi gọi tool. Đây là nguyên tắc quan trọng để tránh model giả mạo danh tính hoặc venue.^[19]

2.6.2. Thanh toán Stripe

Stripe cung cấp PaymentIntent, Payment Sheet và webhook. Backend tạo yêu cầu thanh toán, Flutter hiển thị giao diện native, còn webhook server-to-server xác nhận kết quả trước khi cập nhật trạng thái booking/order. Việc cập nhật phải có idempotency và không tin hoàn toàn vào phản hồi phía client.^[8]

2.6.3. Phạm vi prototype VNPAY và Docker Compose

Mã nguồn có API Python, hợp đồng gRPC và Java service thử nghiệm tạo tham số/chữ ký VNPAY. Tuy nhiên chưa có `'VNPAY_TMN_CODE'` và `'VNPAY_HASH_SECRET'`, nên không thể tạo giao dịch hoặc xác minh callback sandbox. Docker Compose chỉ đóng gói Java service này để nghiên cứu, không phải cấu hình triển khai toàn hệ thống và không chứng minh kiến trúc microservices.

2.7. Streaming phản hồi AI và giao tiếp thời gian thực

2.7.1. Server-Sent Events (SSE) trong streaming AI

Server-Sent Events (SSE) là cơ chế cho phép server đẩy dữ liệu một chiều đến client qua kết nối HTTP duy trì. Khác với polling (client hỏi theo định kỳ) hoặc WebSocket (hai chiều), SSE phù hợp cho bài toán streaming phản hồi AI vì server sinh token liên tục và cần gửi ngay cho client mà không cần nhận phản hồi ngược. Kết nối SSE dùng Content-Type: text/event-stream; client tự động kết nối lại khi mất kết nối.^[1]

Trong đề tài, mỗi khi LLM sinh token mới từ Ollama, backend forward token đó đến Flutter client qua SSE. Flutter dùng Dio để duy trì kết nối SSE và cập nhật state theo Riverpod khi nhận từng chunk. Trải nghiệm người dùng được cải thiện đáng kể so với

chờ toàn bộ câu trả lời: người dùng thấy chữ xuất hiện dần như đang gõ, giảm cảm giác chờ đợi.

2.7.2. WebSocket cho chat và thông báo realtime

WebSocket cung cấp kênh giao tiếp hai chiều, full-duplex qua một TCP connection duy trì. Sau handshake HTTP ban đầu (Upgrade: websocket), cả client và server có thể gửi message bất kỳ lúc nào mà không cần request mới. Độ trễ thấp hơn nhiều so với HTTP polling, phù hợp cho chat realtime và thông báo tức thì.^[1]

Hệ thống dùng WebSocket cho hai tính năng: (1) Chat giữa khách hàng và nhân viên — khi nhân viên chấp nhận yêu cầu, một phòng chat WebSocket được mở giữa hai bên; (2) Thông báo vận hành — khi có booking mới, yêu cầu từ khách hoặc cập nhật trạng thái, backend broadcast thông báo qua Redis pub/sub đến các WebSocket connection đang online. Redis đóng vai trò message broker để đảm bảo thông báo được gửi đến tất cả instance trong trường hợp backend scale ngang.

2.7.3. Thiết kế xử lý bất đồng bộ trong backend AI

FastAPI sử dụng mô hình async/await dựa trên event loop của Python (asyncio). Khi một request gọi LLM qua Ollama hoặc truy vấn Neo4j, coroutine không block event loop mà trả quyền điều khiển để xử lý request khác. Thiết kế này cho phép backend phục vụ nhiều chat session đồng thời mà không cần tạo thread mới cho mỗi request.^[1]

Điểm quan trọng trong thiết kế async của AI pipeline: mỗi bước trong vòng lặp agent (gọi LLM, gọi tool, gọi service, gọi repository) đều được khai báo async. AgentExecutor của LangChain hỗ trợ chế độ ainvoke (async invoke), cho phép toàn bộ vòng lặp agent chạy bất đồng bộ. Khi streaming, generator async yield từng token và SSE handler đọc generator để forward đến client, đảm bảo không có bước nào block vòng lặp chính.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Mô tả bài toán

Hệ thống chatbot AI cho quán thể thao được xây dựng nhằm hỗ trợ toàn bộ quy trình phục vụ tại các cơ sở kinh doanh bida, pickleball và cầu lông. Khách hàng có thể đăng ký, đăng nhập, chọn cơ sở, kiểm tra tài nguyên còn trống, đặt sân hoặc bàn theo khung giờ, xem lịch sử, xác nhận check-in, gọi món, theo dõi đơn hàng, thanh toán và gửi yêu cầu hỗ trợ cho nhân viên. Ngoài giao diện chức năng thông thường, khách hàng có thể diễn đạt yêu cầu bằng ngôn ngữ tự nhiên qua chatbot. AI Agent phân tích ý định, lựa chọn tool được đăng ký và chuyển dữ liệu có cấu trúc đến lớp nghiệp vụ để thực hiện.

Đối với nhu cầu tìm hiểu kiến thức, hệ thống cung cấp nội dung về luật, kỹ thuật, dụng cụ và khái niệm của ba môn thể thao. Dữ liệu này được tổ chức trong Neo4j dưới dạng Knowledge Graph. Luồng Graph RAG kết hợp full-text, keyword, vector search và mở rộng quan hệ để thu thập ngữ cảnh trước khi LLM tổng hợp câu trả lời. LLM không trực tiếp truy vấn cơ sở dữ liệu hoặc tự thay đổi trạng thái nghiệp vụ; mọi hành động đều đi qua tool, service, repository và các quy tắc kiểm tra quyền, tính hợp lệ, xung đột lịch.

Nhân viên tiếp nhận yêu cầu hỗ trợ, trao đổi với khách, xử lý booking, đơn hàng và xem camera theo phạm vi được phân công. Quản trị viên quản lý cơ sở, khu vực, tài nguyên, giá, menu, nhân viên, booking, hóa đơn, camera và thống kê vận hành. Đối tác quản lý cửa hàng, sản phẩm và vòng đời đơn giao đến vị trí của khách trong venue. Dữ liệu nghiệp vụ được lưu bền vững trong PostgreSQL; Redis phục vụ cache, session và realtime; Neo4j lưu tri thức.

Backend là một ứng dụng FastAPI monolith phân tầng, không phải hệ thống microservices. Docker Compose trong mã nguồn chỉ phục vụ Java service thử nghiệm tích hợp VNPAY. Do chưa có mã merchant và khóa bí mật VNPAY, module này chỉ có giá trị nghiên cứu, chưa thể được xem là chức năng thanh toán sandbox hoàn chỉnh. Thanh toán có thể kiểm thử trong phạm vi đồ án là Stripe test mode theo cấu hình môi trường.

3.1.1. Các tác nhân của hệ thống

Tác nhân	Vai trò	Phạm vi tương tác
Customer	Khách hàng	Chat AI, tra cứu tri thức, booking, menu, order, thanh toán, gọi nhân viên và partner order
Staff	Nhân viên venue	Tiếp nhận yêu cầu, chat/call, booking, order, thông báo và camera được phân công
Admin	Quản trị viên	Quản lý venue, tài nguyên, menu, booking, hóa đơn, nhân viên, camera và thống kê
Partner	Đối tác F&B	Quản lý cửa hàng, menu đối tác và trạng thái đơn hàng
AI/LLM provider	Service suy luận	Hiểu yêu cầu, sinh tool call và tổng hợp câu trả lời
Stripe	Cổng thanh toán	Tạo/xác nhận PaymentIntent, Checkout và webhook
VNPay prototype	Service nghiên cứu	Tạo tham số/chữ ký thử nghiệm; chưa đủ cấu hình sandbox

Bảng 3.1. Các tác nhân của hệ thống

3.1.2. Yêu cầu chức năng

Mã	Nhóm chức năng	Yêu cầu
FR01	Tài khoản	Đăng ký, đăng nhập, xác minh token, đổi mật khẩu và xem hồ sơ
FR02	Chat AI	Hội thoại nhiều lượt, phản hồi thường/stream và lưu lịch sử theo session
FR03	Tri thức	Truy vấn luật, kỹ thuật, dụng cụ và khái niệm của ba môn
FR04	Booking	Xem availability, tạo, xem, hủy, check-in, hóa đơn và đổi lịch theo quyền
FR05	Menu/Order	Xem menu, gợi ý món, tạo đơn, xem đơn và cập nhật trạng thái
FR06	Staff Request	Tạo, tiếp nhận, hoàn thành, hủy yêu cầu và trao đổi realtime
FR07	Thông báo	Phát sự kiện theo role/venue, xem và đánh dấu đã đọc
FR08	Thanh toán	Stripe test mode; ghi nhận PaymentTransaction; VNPay chỉ là prototype
FR09	Venue	Quản lý business, venue, area, resource, trạng thái và giá
FR10	Nhân sự	Quản lý staff và phân công theo venue, area hoặc resource
FR11	Camera	Admin quản lý camera; staff chỉ xem camera thuộc phạm vi
FR12	Quản trị	Dashboard, recent activity, booking, order, menu, analytics và hóa đơn
FR13	Đối tác	Quản lý store, partner menu và partner order
FR14	AI orchestration	Intent routing, tool registry, context an toàn, cache và fallback

Bảng 3.2. Yêu cầu chức năng

3.1.3. Yêu cầu phi chức năng

Thuộc tính	Yêu cầu
Bảo mật	Hash mật khẩu, JWT, role/venue scope, validate input, bảo vệ webhook và không cho LLM tự sinh danh tính
Toàn vẹn	Business rule tại service, transaction cho thao tác ghi và kiểm tra xung đột booking
Hiệu suất	Async I/O, Redis cache, intent routing, SSE/WebSocket và index dữ liệu
Khả dụng	Fallback có kiểm soát khi LLM, vector search hoặc dịch vụ ngoài gặp lỗi
Bảo trì	Tách router, schema, service, repository, agent tool và infrastructure client
Mở rộng	Cho phép thêm venue, resource, tool, node/relationship và provider qua cấu hình
Quan sát	Log request, tool đã dùng, metadata, lỗi truy hồi và trạng thái đồng bộ embedding
Trải nghiệm	Flutter phản hồi trạng thái tải/lỗi rõ ràng và điều hướng theo vai trò

Bảng 3.3. Yêu cầu phi chức năng

3.2. Thiết kế cơ sở dữ liệu

3.2.1. Các thực thể và thuộc tính

Tên trường	Kiểu dữ liệu	Chú thích
id	UUID	Khóa chính người dùng
phone	String	Số điện thoại đăng nhập, duy nhất
name	String	Tên hiển thị
email	String, nullable	Email người dùng
stripe_customer_id	String, nullable	Mã customer phía Stripe
password_hash	String	Mật khẩu đã băm
business_id	UUID, nullable	Business mà tài khoản trực thuộc
default_venue_id	UUID, nullable	Venue mặc định
role	Enum	CUSTOMER, STAFF, ADMIN hoặc PARTNER
created_at, updated_at	DateTime	Thời điểm tạo và cập nhật

Bảng 3.4. Users (Người dùng): lưu danh tính, xác thực và vai trò

Tên trường	Kiểu dữ liệu	Chú thích
id	UUID	Khóa chính business
name	String	Tên đơn vị kinh doanh
slug	String	Định danh URL, duy nhất
owner_user_id	UUID	Người dùng sở hữu
is_active	Boolean	Trạng thái hoạt động
created_at, updated_at	DateTime	Thời điểm tạo và cập nhật

Bảng 3.5. *Businesses (Đơn vị kinh doanh): nhóm các venue cùng chủ sở hữu*

Tên trường	Kiểu dữ liệu	Chú thích
id	UUID	Khóa chính venue
business_id	UUID	Khóa ngoại đến Businesses
name	String	Tên cơ sở
address	String	Địa chỉ
timezone	String	Múi giờ vận hành
is_active	Boolean	Trạng thái hoạt động
is_deleted, deleted_at	Boolean, DateTime	Xóa mềm
created_at, updated_at	DateTime	Thời điểm tạo và cập nhật

Bảng 3.6. *Venues (Cơ sở): lưu địa điểm vận hành*

Tên trường	Kiểu dữ liệu	Chú thích
id	UUID	Khóa chính khu vực
venue_id	UUID	Khóa ngoại đến Venues
name	String	Tên khu vực
sort_order	Integer	Thứ tự hiển thị
created_at, updated_at	DateTime	Thời điểm tạo và cập nhật

Bảng 3.7. *VenueAreas (Khu vực): phân chia khu vực trong venue*

Tên trường	Kiểu dữ liệu	Chú thích
id	UUID	Khóa chính tài nguyên
venue_id	UUID	Khóa ngoại đến Venues
area_id	UUID, nullable	Khóa ngoại đến VenueAreas
code	String	Mã tài nguyên trong venue
name	String	Tên hiển thị
resource_type	String	Loại tài nguyên
sport_type	String, nullable	Bida, pickleball hoặc cầu lông
number	Integer, nullable	Số sân/bàn
capacity	Integer	Sức chứa
status	Enum	Trạng thái sẵn sàng/bảo trì/ngừng hoạt động
metadata	JSONB	Thuộc tính mở rộng
hourly_rate	Decimal	Đơn giá theo giờ
is_deleted, deleted_at	Boolean, DateTime	Xóa mềm

Bảng 3.8. *ServiceResources (Tài nguyên phục vụ): lưu sân, bàn hoặc tài nguyên có thể đặt*

Tên trường	Kiểu dữ liệu	Chú thích
id	UUID	Khóa chính phân công
staff_id	UUID	Khóa ngoại đến Users
venue_id	UUID	Venue được phân công
area_id	UUID, nullable	Khu vực được phân công
resource_id	UUID, nullable	Tài nguyên được phân công
scope	Enum	venue, area hoặc resource
starts_at, ends_at	DateTime, nullable	Hiệu lực phân công
is_active	Boolean	Phân công đang hoạt động

Bảng 3.9. *StaffAssignments (Phân công nhân viên): giới hạn phạm vi vận hành của staff*

Tên trường	Kiểu dữ liệu	Chú thích
id	UUID	Khóa chính booking
user_id	UUID	Khách hàng tạo booking
venue_id	UUID	Venue cung cấp dịch vụ
resource_id	UUID	Tài nguyên được đặt
resource_label	String	Tên snapshot của tài nguyên
court_type, court_number	String, Integer	Loại và số sân/bàn
start_time, end_time	DateTime	Khoảng thời gian sử dụng
status	Enum	confirmed, checked_in, cancelled hoặc completed
payment_status	String	Trạng thái thanh toán
notes	Text, nullable	Ghi chú của khách
total_price	Decimal	Tổng tiền booking
checkin_token	String, nullable	Mã xác nhận check-in
checked_in_at	DateTime, nullable	Thời điểm check-in
checked_in_by	UUID, nullable	Người xác nhận check-in
created_at, updated_at	DateTime	Thời điểm tạo và cập nhật

Bảng 3.10. Bookings (Đặt sân/bàn): lưu giao dịch đặt tài nguyên theo thời gian

Tên trường	Kiểu dữ liệu	Chú thích
id	UUID	Khóa chính menu item
venue_id	UUID	Khóa ngoại đến Venues
name	String	Tên món hoặc dịch vụ
category_key, category_name	String	Mã và tên danh mục
description	Text, nullable	Mô tả
unit	String	Đơn vị tính
price	Decimal	Đơn giá
image_url	String, nullable	Đường dẫn hình ảnh
tags	JSONB	Nhãn phục vụ gợi ý
sales_count	Integer	Số lượng đã bán
is_available	Boolean	Còn phục vụ hay không
is_deleted, deleted_at	Boolean, DateTime	Xóa mềm

Bảng 3.11. MenuItems (Món/dịch vụ): lưu menu riêng của từng venue

Tên trường	Kiểu dữ liệu	Chú thích
id	UUID	Khóa chính order
user_id	UUID	Khách tạo đơn
booking_id	UUID, nullable	Booking liên quan
venue_id	UUID	Venue xử lý đơn
resource_id	UUID, nullable	Sân/bàn nhận đơn
resource_label	String, nullable	Tên snapshot tài nguyên
table_number	String, nullable	Số bàn/vị trí
status	Enum	pending, preparing, ready, delivered hoặc cancelled
payment_status	String	Trạng thái thanh toán
total_price	Decimal	Tổng tiền
notes	Text, nullable	Ghi chú
created_at, updated_at	DateTime	Thời điểm tạo và cập nhật

Bảng 3.12. *Orders (Đơn tại venue): lưu đơn món/dịch vụ của khách*

Tên trường	Kiểu dữ liệu	Chú thích
id	UUID	Khóa chính chi tiết
order_id	UUID	Khóa ngoại đến Orders
item_name	String	Tên món tại thời điểm đặt
quantity	Integer	Số lượng
unit_price	Decimal	Đơn giá tại thời điểm đặt

Bảng 3.13. *OrderItems (Chi tiết đơn): lưu snapshot món và giá tại thời điểm đặt*

Tên trường	Kiểu dữ liệu	Chú thích
id	UUID	Khóa chính giao dịch
order_id	UUID, nullable	Đơn hàng liên quan
vnp_transaction_no	String, nullable	Mã giao dịch VNPay nếu có
amount	Decimal	Số tiền
order_type	String	Loại đối tượng thanh toán
status	String	Trạng thái giao dịch
response_code	String, nullable	Mã phản hồi cổng thanh toán
bank_code	String, nullable	Mã ngân hàng
paid_at	DateTime, nullable	Thời điểm thanh toán
created_at, updated_at	DateTime	Thời điểm tạo và cập nhật

Bảng 3.14. *PaymentTransactions (Giao dịch thanh toán): lưu kết quả giao dịch cổng thanh toán*

Tên trường	Kiểu dữ liệu	Chú thích
id	UUID	Khóa chính yêu cầu
user_id, user_name	UUID, String	Khách gửi yêu cầu
venue_id	UUID	Venue tiếp nhận
resource_id, resource_label	UUID, String, nullable	Tài nguyên/vị trí
request_type	Enum	order, payment, help, maintenance, arrival, schedule_change hoặc other
description	Text	Nội dung yêu cầu
table_number	String, nullable	Số bàn/vị trí
status	Enum	pending, accepted, completed hoặc cancelled
accepted_by, accepted_by_name	UUID, String, nullable	Nhân viên tiếp nhận
accepted_at, completed_at	DateTime, nullable	Mốc xử lý
created_at, updated_at	DateTime	Thời điểm tạo và cập nhật

Bảng 3.15. *StaffRequests (Yêu cầu nhân viên): lưu yêu cầu hỗ trợ và vòng đời xử lý*

Tên trường	Kiểu dữ liệu	Chú thích
id	UUID	Khóa chính thông báo
event_type	String	Loại sự kiện
title	String	Tiêu đề
message	Text	Nội dung
target_roles	JSONB	Danh sách role nhận
source	String	Nguồn phát sự kiện
payload	JSONB	Dữ liệu mở rộng
read_at	DateTime, nullable	Thời điểm đã đọc
created_at, updated_at	DateTime	Thời điểm tạo và cập nhật

Bảng 3.16. *Notifications (Thông báo): lưu sự kiện gửi đến người dùng theo vai trò*

Tên trường	Kiểu dữ liệu	Chú thích
id	UUID	Khóa chính camera
venue_id	UUID	Venue lắp đặt
resource_id	UUID, nullable	Tài nguyên được quan sát
name	String	Tên camera
ip_address, port	String, Integer	Địa chỉ kết nối
username, password	String	Thông tin xác thực
camera_brand	Enum	Hãng/chuẩn camera
rtsp_url_override	String, nullable	URL RTSP tùy chỉnh
is_active	Boolean	Trạng thái hoạt động
is_deleted, deleted_at	Boolean, DateTime	Xóa mềm
created_at, updated_at	DateTime	Thời điểm tạo và cập nhật

Bảng 3.17. *Cameras (Camera giám sát): lưu cấu hình camera theo venue/resource*

Tên trường	Kiểu dữ liệu	Chú thích
id	UUID	Khóa chính cửa hàng
owner_user_id	UUID	Tài khoản Partner sở hữu
venue_id	UUID	Venue phục vụ
name, description	String, Text	Tên và mô tả cửa hàng
category	String	Loại cửa hàng
logo_url, phone, address	String, nullable	Thông tin liên hệ
status, is_open	Enum, Boolean	Trạng thái duyệt và mở cửa
rating, total_orders	Decimal, Integer	Điểm và số đơn
delivery_fee	Decimal	Phí giao
estimated_delivery_minutes	Integer	Thời gian giao dự kiến
created_at, updated_at	DateTime	Thời điểm tạo và cập nhật

Bảng 3.18. PartnerStores (Cửa hàng đối tác): lưu thông tin cửa hàng F&B

Tên trường	Kiểu dữ liệu	Chú thích
id	UUID	Khóa chính sản phẩm
store_id	UUID	Khóa ngoại đến PartnerStores
name, description	String, Text	Tên và mô tả
price	Decimal	Đơn giá
category	String	Danh mục
image_url	String, nullable	Hình ảnh
is_available	Boolean	Còn bán hay không
sales_count	Integer	Số lượng đã bán
is_deleted, deleted_at	Boolean, DateTime	Xóa mềm
created_at, updated_at	DateTime	Thời điểm tạo và cập nhật

Bảng 3.19. PartnerMenuItems (Menu đối tác): lưu sản phẩm của cửa hàng đối tác

Tên trường	Kiểu dữ liệu	Chú thích
id	UUID	Khóa chính đơn
store_id	UUID	Cửa hàng xử lý
customer_user_id	UUID	Khách đặt
customer_name, customer_phone	String	Thông tin giao nhận
venue_id	UUID	Venue nhận hàng
delivery_location	String	Vị trí giao trong venue
status	Enum	pending đến delivered/cancelled
payment_status	String	Trạng thái thanh toán
subtotal, delivery_fee, total_price	Decimal	Giá trị đơn
notes	Text, nullable	Ghi chú
created_at, updated_at	DateTime	Thời điểm tạo và cập nhật

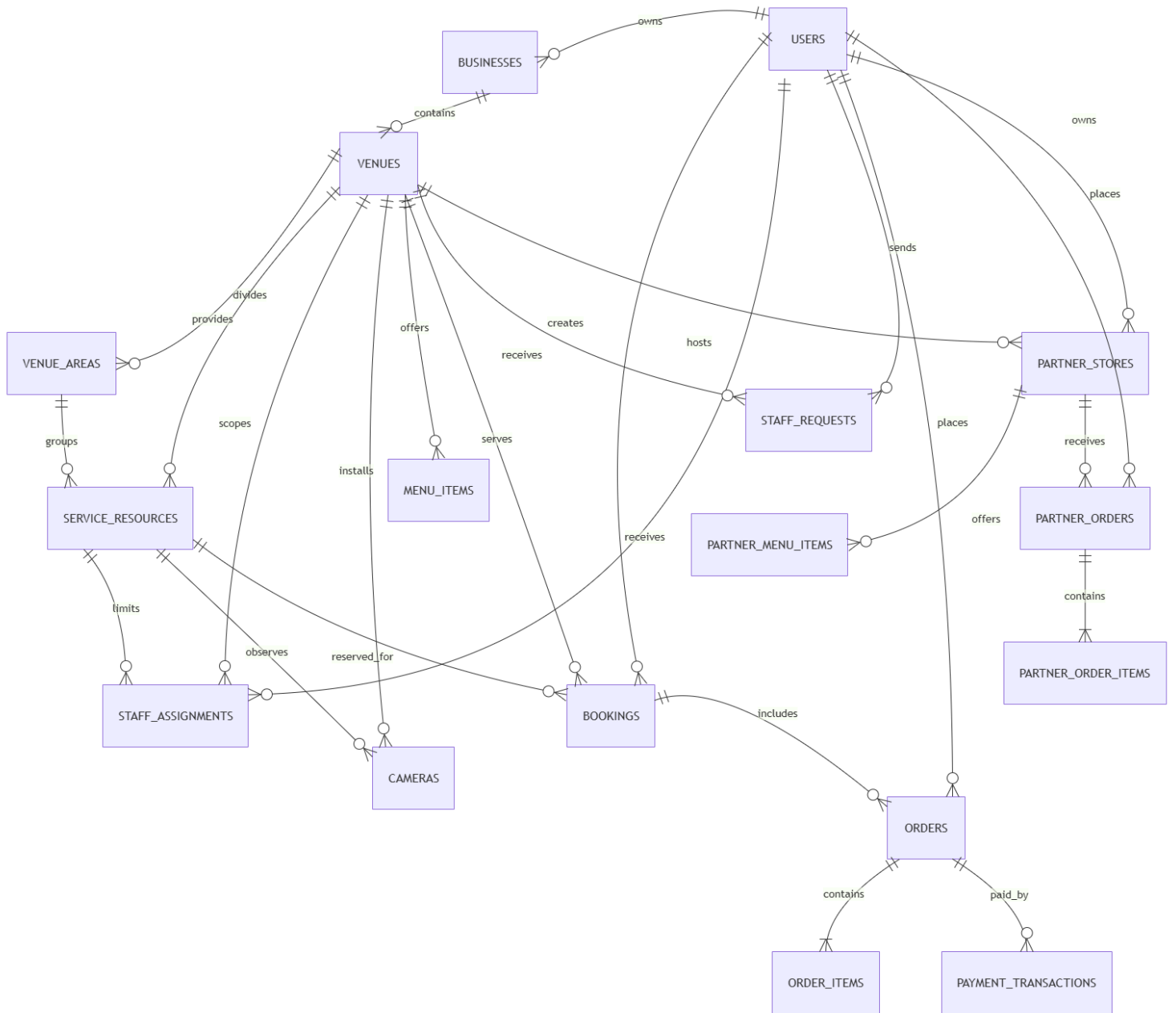
Bảng 3.20. *PartnerOrders (Đơn đối tác): lưu đơn giao từ partner store*

Tên trường	Kiểu dữ liệu	Chú thích
id	UUID	Khóa chính chi tiết
order_id	UUID	Khóa ngoại đến PartnerOrders
item_name	String	Tên sản phẩm tại thời điểm đặt
quantity	Integer	Số lượng
unit_price	Decimal	Đơn giá tại thời điểm đặt

Bảng 3.21. *PartnerOrderItems (Chi tiết đơn đối tác): lưu snapshot sản phẩm trong đơn*

3.2.2. Xây dựng mô hình thực thể liên kết

Mô hình ER tập trung vào các quan hệ tạo nên quy trình vận hành: business quản lý nhiều venue; venue chứa area, resource, menu và camera; customer tạo booking, order, staff request và partner order; staff được giới hạn bằng assignment. OrderItem và PartnerOrderItem là thực thể chi tiết nhằm biểu diễn quan hệ một-nhiều và bảo toàn snapshot giá.



Hình 3.1. Sơ đồ mô hình thực thể liên kết

3.2.3. Chuyển từ mô hình thực thể liên kết sang mô hình quan hệ

Sau khi chuyển đổi, mỗi thực thể mạnh trở thành một quan hệ; quan hệ một-nhiều được biểu diễn bằng khóa ngoại ở phía nhiều. Các quan hệ chi tiết được tách thành bảng riêng. Ký hiệu PK chỉ khóa chính, FK chỉ khóa ngoại.

- Users(id PK, phone, name, email, stripe_customer_id, password_hash, business_id FK, default_venue_id FK, role, created_at, updated_at).
- Businesses(id PK, name, slug, owner_user_id FK, is_active, created_at, updated_at).

- Venues(id PK, business_id FK, name, address, timezone, is_active, is_deleted, deleted_at, created_at, updated_at).
- VenueAreas(id PK, venue_id FK, name, sort_order, created_at, updated_at).
- ServiceResources(id PK, venue_id FK, area_id FK, code, name, resource_type, sport_type, number, capacity, status, metadata, hourly_rate).
- StaffAssignments(id PK, staff_id FK, venue_id FK, area_id FK, resource_id FK, scope, starts_at, ends_at, is_active).
- Bookings(id PK, user_id FK, venue_id FK, resource_id FK, resource_label, court_type, court_number, start_time, end_time, status, payment_status, total_price, checkin_token).
- MenuItems(id PK, venue_id FK, name, category_key, category_name, description, unit, price, image_url, tags, sales_count, is_available).
- Orders(id PK, user_id FK, booking_id FK, venue_id FK, resource_id FK, resource_label, table_number, status, payment_status, total_price, notes).
- OrderItems(id PK, order_id FK, item_name, quantity, unit_price).
- PaymentTransactions(id PK, order_id FK, transaction_no, amount, order_type, status, response_code, bank_code, paid_at).
- StaffRequests(id PK, user_id FK, venue_id FK, resource_id FK, request_type, description, status, accepted_by FK, accepted_at, completed_at).
- Notifications(id PK, event_type, title, message, target_roles, source, payload, read_at, created_at, updated_at).
- Cameras(id PK, venue_id FK, resource_id FK, name, ip_address, port, username, password, camera_brand, rtsp_url_override, is_active).
- PartnerStores(id PK, owner_user_id FK, venue_id FK, name, category, status, is_open, rating, delivery_fee, estimated_delivery_minutes).
- PartnerMenuItems(id PK, store_id FK, name, description, price, category, image_url, is_available, sales_count).
- PartnerOrders(id PK, store_id FK, customer_user_id FK, venue_id FK, delivery_location, status, payment_status, subtotal, delivery_fee, total_price).
- PartnerOrderItems(id PK, order_id FK, item_name, quantity, unit_price).

3.2.4. Chuẩn hóa

Các bảng được đánh giá theo 1NF, 2NF và 3NF. Mỗi cột nghiệp vụ chính chứa giá trị nguyên tử; mỗi bảng dùng khóa UUID đơn nên không phát sinh phụ thuộc bộ phận; thuộc tính không khóa phụ thuộc vào khóa chính của chính bảng. JSONB chỉ được dùng cho metadata/tags/payload mở rộng, không thay thế các quan hệ nghiệp vụ cần truy vấn và ràng buộc thường xuyên.

Bảng/nhóm bảng	Phân tích chuẩn hóa	Kết luận
Users, Businesses, Venues	Mỗi bảng mô tả một chủ thể; thuộc tính phụ thuộc trực tiếp vào id	3NF
VenueAreas, ServiceResources	Tách area/resource khỏi venue để tránh lặp thông tin cơ sở	3NF
StaffAssignments	Tách quan hệ phân công và thời hạn khỏi Users	3NF
Bookings	Tham chiếu user/venue/resource; snapshot resource_label là phi chuẩn hóa có chủ đích để lưu lịch sử	3NF thực dụng
MenuItems	Menu thuộc venue; tags là metadata hỗ trợ tìm kiếm	3NF thực dụng
Orders, OrderItems	Tách danh sách món thành OrderItems; tên và giá là snapshot giao dịch	3NF thực dụng
PaymentTransactions	Tách trạng thái giao dịch khỏi order để hỗ trợ nhiều lần thanh toán/đối soát	3NF
StaffRequests	Tách yêu cầu khỏi booking/order; accepted_by tham chiếu Users	3NF
Notifications	Payload JSONB phục vụ nhiều loại sự kiện; dữ liệu cốt lõi vẫn nguyên tử	3NF thực dụng
Cameras	Tách cấu hình thiết bị khỏi resource; venue/resource chỉ lưu khóa ngoại	3NF

PartnerStores, PartnerMenuItems	Tách cửa hàng và sản phẩm để tránh lặp thông tin store	3NF
PartnerOrders, PartnerOrderItems	Tách danh sách sản phẩm; snapshot giá bảo toàn lịch sử	3NF thực dụng

Bảng 3.22. Chuẩn hoá

“3NF thực dụng” không phải thuật ngữ chuẩn trong lý thuyết CSDL. Em dùng nó để chỉ mô hình gần đạt 3NF nhưng cố ý giữ một số dữ liệu lặp hoặc dữ liệu tổng hợp vì yêu cầu vận hành.

- Ví dụ OrderItems lưu:

order_id

item_name

quantity

unit_price

item_name và *unit_price* có thể lấy từ MenuItems, nhưng hệ thống vẫn lưu lại vì đây là snapshot tại thời điểm đặt hàng. Nếu sau này menu đổi tên hoặc đổi giá, hóa đơn cũ vẫn phải giữ nguyên.

Tương tự:

- **Booking**.resource_label
- **Order**.resource_label
- **StaffRequest**.user_name
- **StaffRequest**.accepted_by_name
- **PartnerOrder**.customer_name
- **PartnerOrder**.customer_phone

Các trường này cố ý sao chép dữ liệu để:

- Giữ lịch sử giao dịch.
- Tránh hóa đơn cũ thay đổi theo dữ liệu hiện tại.
- Hiện thị được thông tin kể cả khi bản ghi gốc bị xóa mềm.
- Giảm một số phép JOIN khi đọc.

Kết luận: Mô hình dữ liệu chủ yếu được chuẩn hóa đến 3NF. Một số thuộc tính snapshot và JSONB được giữ có chủ đích để bảo toàn lịch sử giao dịch, hỗ trợ dữ liệu mở rộng

và tối ưu truy xuất. Đây là quyết định phi chuẩn hóa có kiểm soát, không thay thế các quan hệ nghiệp vụ cốt lõi.

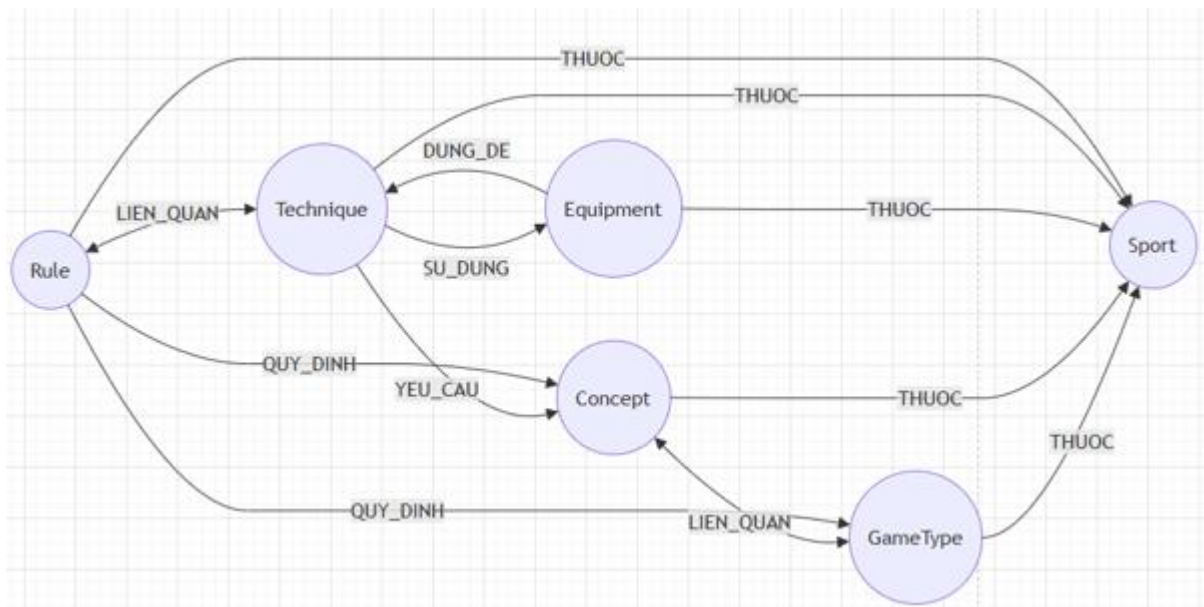
3.2.5. Thiết kế Knowledge Graph Neo4j

Nhãn node	Thuộc tính chính	Vai trò
Sport	name, normalized_name	Môn bida, pickleball hoặc cầu lông
Rule	name, description, embedding	Luật và quy định
Technique	name, description, embedding	Kỹ thuật thực hiện
Equipment	name, description, embedding	Dụng cụ và thiết bị
Concept	name, description, embedding	Khái niệm, thuật ngữ
GameType	name, description, embedding	Biến thể/kiểu thi đấu

Bảng 3.23. Các nhãn node trong Knowledge Graph Neo4j

Relationship	Nguồn - đích	Ý nghĩa
THUOC	KnowledgeEntity → Sport	Thực thể thuộc môn thể thao
LIEN_QUAN	KnowledgeEntity ↔ KnowledgeEntity	Hai nội dung có liên quan
QUY_DINH	Rule → Concept/GameType	Luật quy định một khái niệm hoặc kiểu chơi
SU_DUNG	Technique/GameType → Equipment	Kỹ thuật/kiểu chơi sử dụng dụng cụ
DUNG_DE	Equipment → Technique	Dụng cụ được dùng cho kỹ thuật
YEU_CAU	Technique/GameType → Concept	Điều kiện hoặc khái niệm cần có

Bảng 3.24. Các loại quan hệ trong Knowledge Graph Neo4j



Hình 3.2. Sơ đồ cấu trúc Knowledge Graph

3.3.1. Danh sách tác nhân và ca sử dụng

Customer

- UC01: Đăng ký tài khoản.
- UC02: Đăng nhập và xác minh phiên.
- UC03: Xem hồ sơ và đổi mật khẩu.
- UC04: Chọn venue và xem tài nguyên.
- UC05: Chat với AI Agent.
- UC06: Tra cứu tri thức ba môn thể thao.
- UC07: Kiểm tra lịch trống.
- UC08: Tạo booking.
- UC09: Xem lịch sử, booking đang hoạt động và hóa đơn.
- UC10: Hủy booking hoặc xác nhận check-in.
- UC11: Xem menu và gợi ý món.
- UC12: Tạo và theo dõi order.
- UC13: Gọi nhân viên hoặc gửi yêu cầu hỗ trợ.
- UC14: Chat hoặc gọi với nhân viên.
- UC15: Xem và đánh dấu thông báo.

- UC16: Thanh toán qua Stripe (test mode).
- UC17: Xem cửa hàng và menu đối tác.
- UC18: Tạo partner order.

Staff

- UC19: Xem thông báo và yêu cầu thuộc phạm vi phụ trách.
- UC20: Tiếp nhận, hoàn thành hoặc hủy staff request.
- UC21: Trao đổi chat/call và đóng phòng hỗ trợ.
- UC22: Theo dõi booking và order vận hành.
- UC23: Xem camera được phân công.

Admin

- UC24: Xem dashboard, recent activity và analytics.
- UC25: Quản lý booking, hóa đơn, check-in, đổi lịch và trạng thái.
- UC26: Quản lý order.
- UC27: Quản lý menu và availability.
- UC28: Quản lý venue, area, resource, giá và trạng thái.
- UC29: Quản lý staff và staff assignment.
- UC30: Quản lý camera.
- UC31: Theo dõi thông báo và hoạt động hệ thống.

Partner

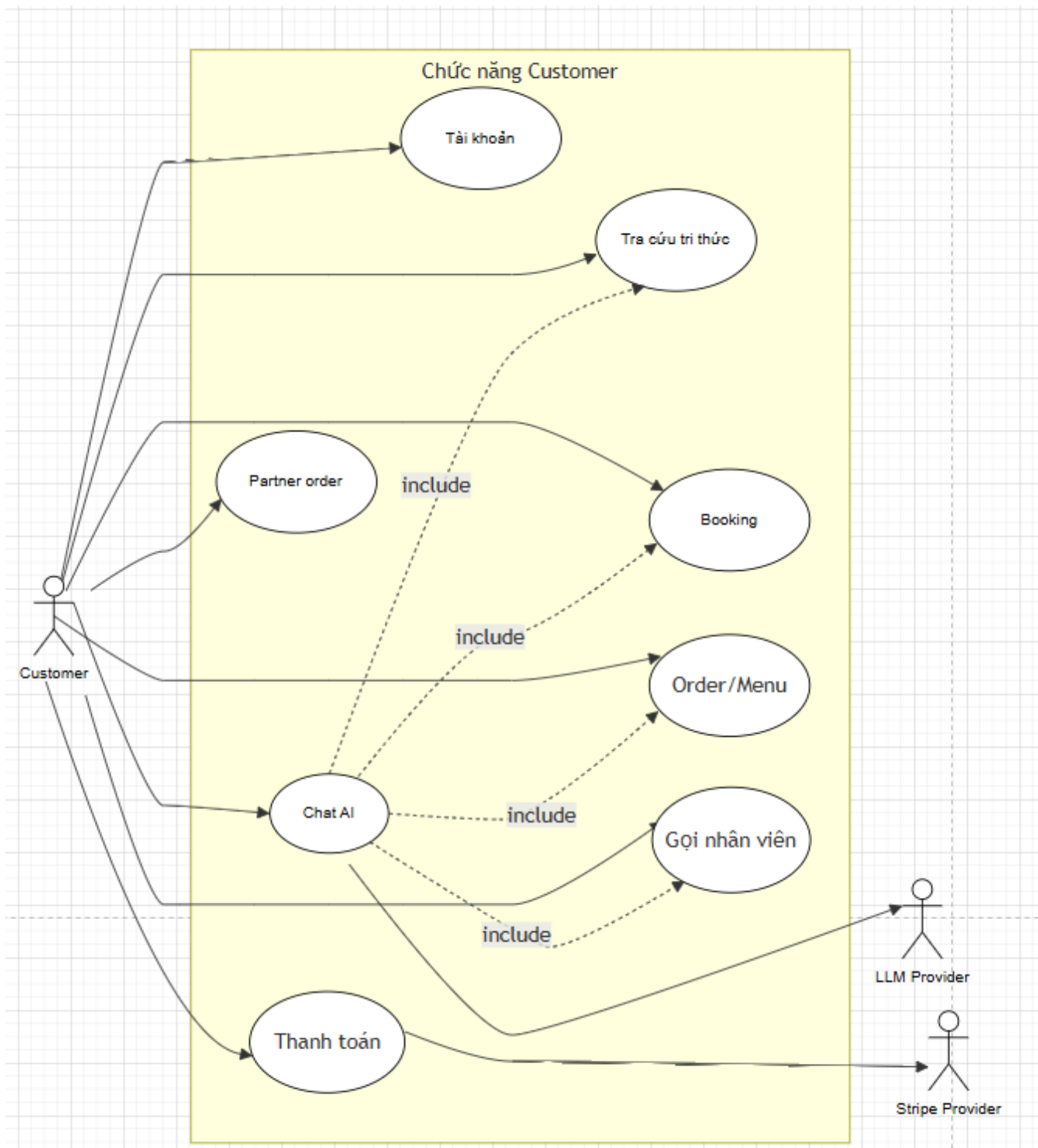
- UC32: Cập nhật thông tin và trạng thái cửa hàng.
- UC33: Quản lý partner menu.
- UC34: Quản lý vòng đời partner order.

Hệ thống

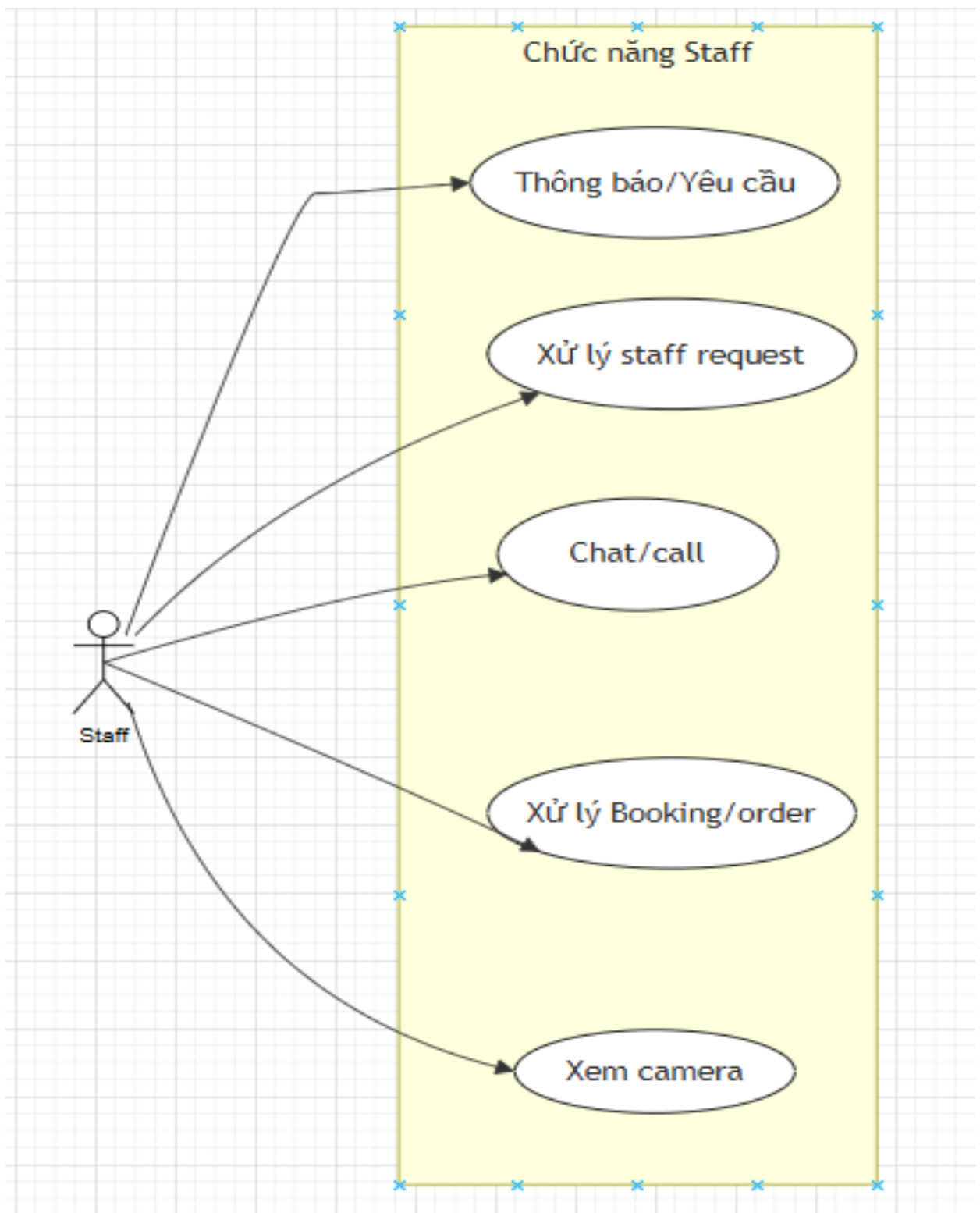
- UC35: Đồng bộ embedding tri thức tăng dần khi backend khởi động.

Prototype nghiên cứu

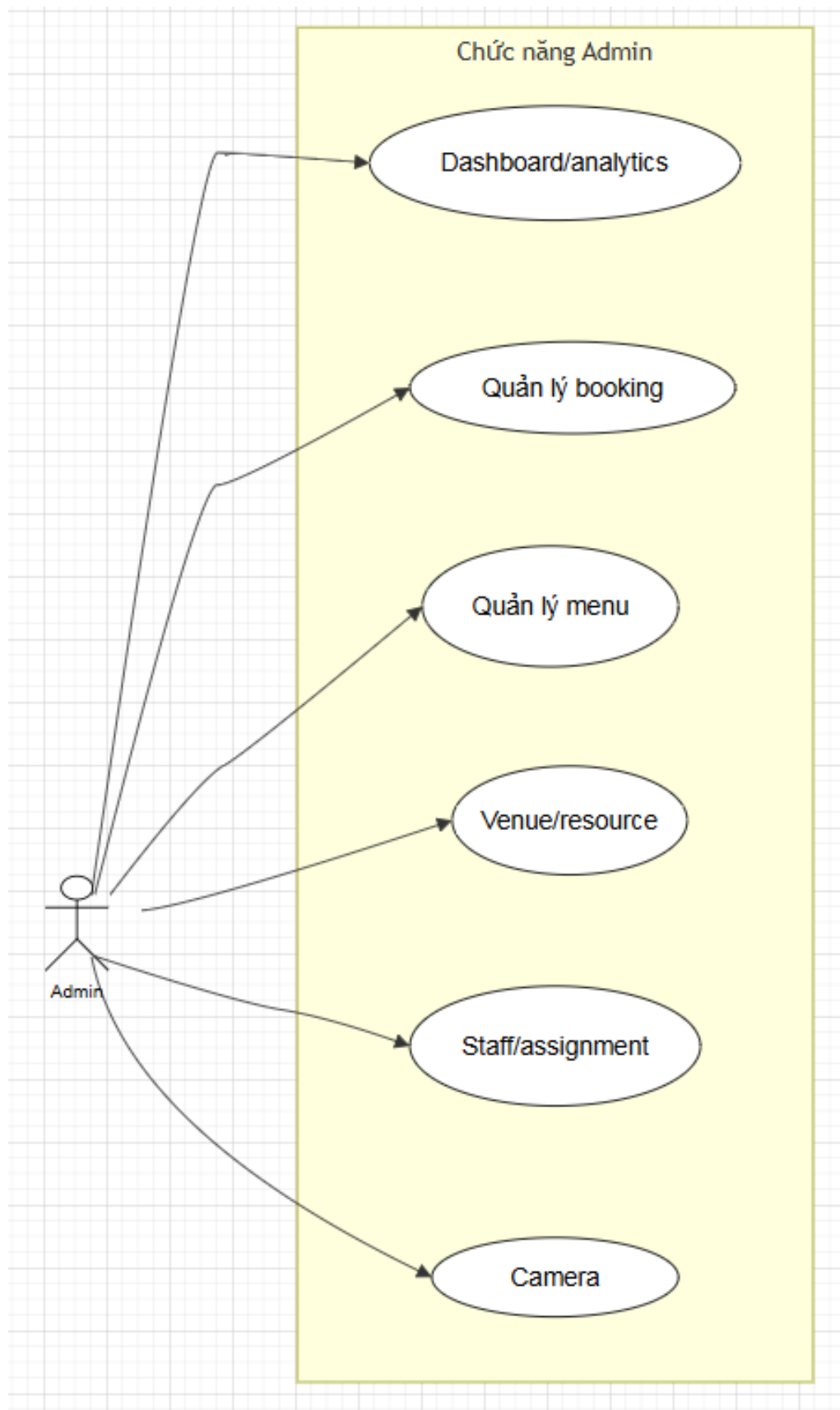
- UC36: Tạo và truy vấn giao dịch VNPay phục vụ nghiên cứu; chưa sử dụng được do hạn chế sandbox.



Hình 3.3. Sơ đồ Use-case tổng quát (Customer)



Hình 3.4. Sơ đồ Use-case tổng quát (Staff)



Hình 3.5. Sơ đồ Use-case tổng quát (Admin/Manager)

3.3.2. Đặc tả các ca sử dụng quan trọng

Thuộc tính	Nội dung
Mã và tên	UC05 - Chat với AI Agent
Tác nhân	Customer; Staff/Admin trong phạm vi cho phép
Mô tả	Tiếp nhận yêu cầu ngôn ngữ tự nhiên và điều phối trả lời hoặc hành động.
Tiền điều kiện	Người dùng đã đăng nhập; backend và ít nhất một provider AI khả dụng.
Luồng chính	1. Flutter gửi message và session_id. 2. ChatService nạp history/context. 3. IntentRouter xác định hướng xử lý. 4. AgentExecutor gửi prompt và tool schema cho LLM. 5. Nếu có tool call, backend thực thi tool. 6. Observation được đưa lại cho LLM. 7. Trả reply, tools_used và metadata.
Luồng thay thế/ngoại lệ	Cache hit thì trả nhanh; thiếu dữ kiện thì hỏi lại; tool/provider lỗi thì dùng fallback hoặc thông báo không thể hoàn tất.
Hậu điều kiện	Lịch sử hội thoại được cập nhật; mọi thay đổi nghiệp vụ chỉ tồn tại nếu service thực thi thành công.

Bảng 3.25. Bảng đặc tả UC05. Chat với AI Agent

Thuộc tính	Nội dung
Mã và tên	UC06 - Tra cứu tri thức ba môn
Tác nhân	Customer hoặc người dùng chatbot
Mô tả	Trả lời luật, kỹ thuật, dụng cụ và khái niệm từ Knowledge Graph.
Tiền điều kiện	Neo4j đã có dữ liệu; full-text index tồn tại; embedding có thể đã được đồng bộ.
Luồng chính	1. Nhận câu hỏi. 2. Phát hiện môn và loại thực thể. 3. Chạy full-text/vector và keyword fallback. 4. Hợp nhất hạng bằng RRF. 5. Chọn seed node và mở rộng quan hệ 1-2 bước. 6. Đóng gói context. 7. LLM tổng hợp câu trả lời.
Luồng thay thế/ngoại lệ	Vector không phù hợp hoặc lỗi thì dùng lexical retrieval; không có kết quả thì trả lời rõ chưa tìm thấy trong kho tri thức.
Hậu điều kiện	Không thay đổi dữ liệu nghiệp vụ; trả kết quả có node, quan hệ và nguồn context.

Bảng 3.26. Bảng đặc tả UC06. Tra cứu tri thức ba môn

Thuộc tính	Nội dung
Mã và tên	UC08 - Tạo booking
Tác nhân	Customer
Mô tả	Đặt một tài nguyên thể thao trong khoảng thời gian xác định.
Tiền điều kiện	Đã đăng nhập; venue/resource đang hoạt động; thời gian hợp lệ.
Luồng chính	1. Chọn venue, loại/số sân và thời gian. 2. Backend xác định resource. 3. Kiểm tra quyền, trạng thái và xung đột lịch. 4. Tính giá. 5. Tạo Booking. 6. Trả mã booking và trạng thái thanh toán.
Luồng thay thế/ngoại lệ	Thiếu dữ liệu thì yêu cầu bổ sung; resource không tồn tại hoặc đã bận thì trả danh sách/thời gian thay thế; lỗi transaction thì không tạo booking.
Hậu điều kiện	Booking được lưu ở trạng thái hợp lệ và có thể xuất hiện trong lịch sử/hóa đơn.

Bảng 3.27. Bảng đặc tả UC08. Tạo booking

Thuộc tính	Nội dung
Mã và tên	UC12 - Tạo và theo dõi order
Tác nhân	Customer
Mô tả	Đặt món/dịch vụ từ menu venue và theo dõi trạng thái xử lý.
Tiền điều kiện	Đã đăng nhập; venue xác định; item còn phục vụ; số lượng lớn hơn 0.
Luồng chính	1. Chọn item và số lượng. 2. Backend lấy giá hiện hành. 3. Kiểm tra booking/vị trí giao nếu cần. 4. Tạo Order và OrderItems trong transaction. 5. Phát thông báo cho staff. 6. Customer theo dõi trạng thái.
Luồng thay thế/ngoại lệ	Item hết phục vụ hoặc không thuộc venue thì loại khỏi đơn và báo lại; lỗi tạo chi tiết thì rollback toàn bộ order.
Hậu điều kiện	Order cùng snapshot tên, số lượng và giá được lưu; staff nhận được yêu cầu.

Bảng 3.28. Bảng đặc tả UC12. Tạo và theo dõi order

Thuộc tính	Nội dung
Mã và tên	UC13 - Gọi nhân viên/yêu cầu hỗ trợ
Tác nhân	Customer
Mô tả	Tạo yêu cầu order, payment, help, maintenance hoặc thay đổi lịch.
Tiền điều kiện	Đã đăng nhập và có venue hiện hành.
Luồng chính	1. Chọn loại yêu cầu/nhập mô tả. 2. Backend gắn user, venue và resource từ context tin cậy. 3. Tạo StaffRequest pending. 4. Phát notification/WebSocket đến staff phù hợp. 5. Trả trạng thái cho khách.
Luồng thay thế/ngoại lệ	Không có staff online vẫn lưu yêu cầu pending; dữ liệu vị trí thiếu thì yêu cầu khách bổ sung.
Hậu điều kiện	Yêu cầu được lưu và có thể chuyển sang accepted, completed hoặc cancelled.

Bảng 3.29. Bảng đặc tả UC13. Gọi nhân viên/yêu cầu hỗ trợ

Thuộc tính	Nội dung
Mã và tên	UC16 - Thanh toán qua Stripe
Tác nhân	Customer và Stripe
Mô tả	Thanh toán booking/order trong Stripe test mode.
Tiền điều kiện	Đối tượng thanh toán tồn tại; số tiền được backend tính; khóa Stripe hợp lệ.
Luồng chính	1. Backend tạo PaymentIntent hoặc Checkout. 2. Flutter hiển thị Payment Sheet/trang thanh toán. 3. Stripe xử lý test payment. 4. Backend xác nhận kết quả và/hoặc nhận webhook. 5. Cập nhật payment status idempotent.
Luồng thay thế/ngoại lệ	Thanh toán thất bại/hủy thì giữ trạng thái chưa thanh toán; webhook trùng không được ghi nhận hai lần; client response không thay thế webhook.
Hậu điều kiện	PaymentTransaction và trạng thái đối tượng được cập nhật nhất quán.

Bảng 3.30. Bảng đặc tả UC16. Thanh toán qua Stripe

Thuộc tính	Nội dung
Mã và tên	UC20 - Xử lý staff request
Tác nhân	Staff
Mô tả	Tiếp nhận và hoàn thành yêu cầu thuộc venue/phạm vi phân công.
Tiền điều kiện	Staff đã đăng nhập, assignment còn hiệu lực và request đang pending/accepted.
Luồng chính	1. Xem danh sách pending/active. 2. Chọn accept. 3. Hệ thống ghi accepted_by và accepted_at. 4. Staff trao đổi hoặc thực hiện yêu cầu. 5. Chọn complete. 6. Hệ thống ghi completed_at và thông báo customer.
Luồng thay thế/ngoại lệ	Request đã được staff khác nhận thì từ chối cập nhật; không đúng phạm vi thì trả lỗi quyền; có thể cancel theo quy tắc.
Hậu điều kiện	Vòng đời request và người xử lý được lưu đầy đủ.

Bảng 3.31. Bảng đặc tả UC20. Xử lý staff request

Thuộc tính	Nội dung
Mã và tên	UC25 - Quản lý booking
Tác nhân	Admin
Mô tả	Xem, check-in, đổi lịch, cập nhật trạng thái và xem hóa đơn booking.
Tiền điều kiện	Admin đã đăng nhập và có quyền trên venue chứa booking.
Luồng chính	1. Lọc danh sách booking. 2. Chọn booking. 3. Thực hiện check-in/reschedule/status update hoặc xem bill. 4. Service kiểm tra quyền và xung đột. 5. Lưu thay đổi và ghi recent activity.
Luồng thay thế/ngoại lệ	Đổi lịch bị trùng thì từ chối; booking đã hoàn thành/hủy chỉ cho phép thao tác phù hợp; venue ngoài phạm vi không được truy cập.
Hậu điều kiện	Booking được cập nhật có kiểm soát và phản ánh trên dashboard.

Bảng 3.32. Bảng đặc tả UC25. Quản lý booking

Thuộc tính	Nội dung
Mã và tên	UC34 - Quản lý vòng đời partner order
Tác nhân	Partner
Mô tả	Xử lý đơn của cửa hàng từ pending đến delivered hoặc cancelled.
Tiền điều kiện	Partner sở hữu store và order thuộc store đó.
Luồng chính	1. Xem đơn. 2. Accept. 3. Chuyển preparing, ready, delivering. 4. Xác nhận delivered. 5. Hệ thống cập nhật tổng đơn và thông báo customer.
Luồng thay thế/ngoại lệ	Partner không sở hữu store thì từ chối; chuyển trạng thái sai thứ tự hoặc đơn đã cancelled/delivered thì không cho cập nhật.
Hậu điều kiện	Trạng thái order phản ánh đúng tiến độ và được customer theo dõi.

Bảng 3.33 Bảng đặc tả UC34. Quản lý vòng đời partner order

3.3.3. Quan hệ use-case và ràng buộc nghiệp vụ

Các ca sử dụng trong hệ thống có một số quan hệ và ràng buộc quan trọng. Đối với chức năng chat với AI Agent (UC05), hệ thống có thể thực hiện thêm các chức năng tra cứu tri thức (UC06), kiểm tra lịch trống và tạo booking (UC07–UC08), hoặc tra cứu menu và tạo yêu cầu hỗ trợ (UC11–UC13) tùy thuộc vào ý định được nhận diện từ nội dung hội thoại của người dùng.

Trong quy trình đặt sân, việc kiểm tra lịch trống (UC07) là điều kiện bắt buộc trước khi tạo booking (UC08). Nếu tài nguyên được chọn đã bị đặt trong khoảng thời gian yêu cầu, hệ thống sẽ từ chối tạo booking mới nhằm đảm bảo tính nhất quán dữ liệu.

Đối với quy trình đặt món, hệ thống luôn thực hiện bước kiểm tra menu, trạng thái khả dụng của món ăn, venue tương ứng và đơn giá hiện hành trước khi tạo order. Các kiểm tra này được thực hiện tại backend nhằm bảo đảm tính chính xác và tránh phụ thuộc vào dữ liệu do phía client cung cấp.

Chức năng thanh toán (UC16) là một luồng mở rộng, chỉ được kích hoạt khi người dùng lựa chọn phương thức thanh toán cho booking hoặc order. Do đó, thanh toán không phải là bước bắt buộc trong mọi trường hợp sử dụng hệ thống.

Chức năng yêu cầu hỗ trợ nhân viên (UC13) có thể được tạo độc lập hoặc được khởi tạo từ ngữ cảnh hiện tại của người dùng, chẳng hạn như trong quá trình đặt sân, đặt món hoặc trao đổi với AI Agent.

Hệ thống áp dụng cơ chế phân quyền theo vai trò. Customer, Staff, Admin và Partner chỉ được phép truy cập các dữ liệu và chức năng thuộc phạm vi được cấp quyền, bao gồm venue, cửa hàng hoặc tài nguyên liên quan.

Để bảo đảm an toàn, các thông tin ngữ cảnh quan trọng như user_id, role và venue_id được lấy từ phiên xác thực hoặc backend context, không được phép sinh tự động từ các tham số do mô hình ngôn ngữ tạo ra.

Ngoài ra, chức năng VNPay (UC36) hiện chỉ được xây dựng dưới dạng nguyên mẫu phục vụ mục đích nghiên cứu. Do hạn chế của môi trường sandbox, chức năng này chưa được tích hợp vào quy trình thanh toán vận hành của hệ thống và không được xem là một luồng nghiệp vụ chính thức.

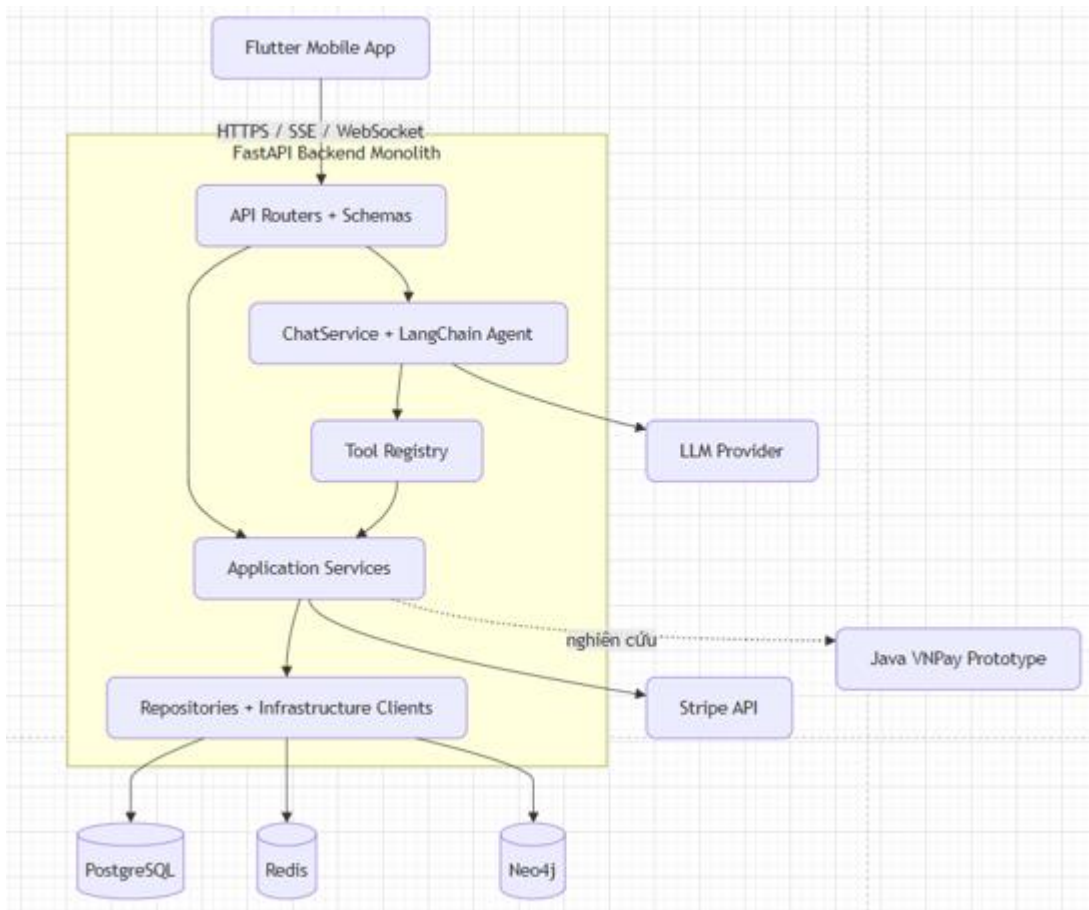
3.4. Thiết kế kiến trúc hệ thống

3.4.1. Kiến trúc monolith phân tầng

Backend FastAPI được triển khai thành một tiến trình ứng dụng monolith. Các module cùng được đóng gói và triển khai, nhưng trách nhiệm được phân tầng rõ: router xử lý giao thức và dependency; schema validate dữ liệu; service thực thi use case; repository làm việc với PostgreSQL; AI Agent điều phối tool; client kết nối Redis, Neo4j, Stripe và provider LLM.

Tầng	Thành phần	Trách nhiệm
Presentation	Flutter/Riverpod	Giao diện, state, route, HTTP, SSE và WebSocket
API	FastAPI router/schema/dependency	Xác thực, validation và hợp đồng response
Application	Booking/Order/Staff/Payment services	Business rule, transaction và điều phối use case
AI	ChatService, IntentRouter, AgentExecutor, tools	Hiểu intent, sinh/routing tool call và tổng hợp kết quả
Data access	Repositories và clients	PostgreSQL, Redis và Neo4j
External	LLM provider, Stripe, VNPay prototype	Dịch vụ tích hợp bên ngoài

Bảng 3.34. Các tầng kiến trúc của hệ thống



Hình 3.6. Sơ đồ kiến trúc tổng thể

3.4.3. Phân bố dữ liệu và trạng thái

Dữ liệu/trạng thái	Nơi lưu	Lý do
User, venue, booking, order, payment	PostgreSQL	Dữ liệu quan hệ cần transaction và ràng buộc
Chat history/session	Redis	Đọc ghi nhanh và có TTL
Intent/result cache	Redis	Giảm gọi embedding/LLM lặp lại
Tri thức và embedding node	Neo4j	Kết hợp search với traversal quan hệ
Agent scratchpad	Bộ nhớ tiến trình	Chỉ tồn tại trong một vòng AgentExecutor
UI state	Riverpod trong Flutter	Quản lý trạng thái màn hình và phiên

Bảng 3.35. Phân bố dữ liệu và trạng thái

3.5. Phân tích và thiết kế hệ thống AI Agent, Tool Calling và Graph RAG

3.5.1. Phân tích bài toán hội thoại và ý định người dùng

AI Agent trong đề tài không phải chatbot chỉ hỏi đáp văn bản. Nó là lớp điều phối giữa ngôn ngữ tự nhiên và các use case của hệ thống. Đầu vào có thể thiếu cấu trúc, chứa nhiều ý định hoặc phụ thuộc lịch sử hội thoại; đầu ra phải hoặc là câu trả lời có căn cứ, hoặc là kết quả nghiệp vụ đã được backend xác nhận. Vì vậy bài toán được tách thành nhận diện ý định, thu thập tham số, chọn tool, thực thi có kiểm soát và tổng hợp phản hồi.

Nhóm ý định	Ví dụ yêu cầu	Hướng xử lý
Knowledge	Luật giao bóng pickleball là gì?	Gọi query_knowledge và dùng Graph RAG
Booking inquiry	Tối mai còn sân cầu lông không?	Gọi tool kiểm tra availability/schedule
Booking action	Đặt sân số 2 từ 19 giờ đến 21 giờ	Thu thập đủ thời gian, resource rồi gọi book_court
Menu inquiry	Có đồ uống nào ít ngọt?	Gọi recommend_menu hoặc truy vấn menu
Order action	Cho tôi hai nước suối	Xác định venue/booking, parse items rồi gọi order_menu_items
Staff support	Gọi nhân viên giúp tôi	Xác định loại yêu cầu rồi gọi call_staff
Account/navigation	Xem lịch sử đặt sân	Trả hướng dẫn hoặc metadata điều hướng phù hợp
General conversation	Chào bạn	LLM trả lời trực tiếp, không gọi tool
Unsafe/unsupported	Yêu cầu ngoài quyền hoặc thiếu dữ liệu	Từ chối, hỏi lại hoặc nêu giới hạn

Bảng 3.36. Các nhóm ý định và hướng xử lý của AI Agent

Một câu có thể chứa nhiều ý định, ví dụ vừa hỏi sân trống vừa yêu cầu đặt. Thiết kế ưu tiên kiểm tra thông tin trước, sau đó mới thực hiện hành động ghi dữ liệu. Đối với thao tác có tác động như booking, order hoặc payment, Agent phải có đủ tham số bắt buộc và không được tự suy đoán user_id, role, venue hoặc giá.

3.5.2. Kiến trúc nội bộ và ranh giới trách nhiệm

Thành phần	Đầu vào	Đầu ra/trách nhiệm
Chat API	HTTP request, JWT	Xác thực và tạo ChatRequest/ChatResponse
ChatService	Message, session, trusted context	Nạp history, gọi router/agent và chuẩn hóa response
IntentRouter	Câu hiện tại và mẫu intent	Intent dự đoán, confidence và đường xử lý nhanh
VenueAgent	Prompt, history, context và tools	Khởi tạo/chọn executor và quản lý fallback
LangChain AgentExecutor	Messages và tool schemas	Vòng lặp LLM → action → observation → final answer
LLM	System prompt, history, schemas	Sinh câu trả lời hoặc tool call có arguments
Tool adapter	Arguments đã parse	Chuyển yêu cầu sang retriever/application service
Application service	DTO và trusted context	Kiểm tra quyền, business rule, transaction và lỗi
Repository/client	Truy vấn đã được kiểm soát	Đọc/ghi PostgreSQL, Redis hoặc Neo4j
Response mapper	Final answer và tool result	Trả reply, tools_used, metadata và UI action

Bảng 3.37. Kiến trúc nội bộ và ranh giới trách nhiệm

Ranh giới quan trọng nhất là giữa suy luận và thực thi. LLM có thể đề xuất hành động và sinh arguments, nhưng AgentExecutor mới là thành phần điều phối gọi tool; tool chuyển tiếp sang service; service mới có quyền quyết định thao tác có hợp lệ và có được ghi dữ liệu hay không.

3.5.3. Thiết kế prompt, memory và ngữ cảnh tin cậy

Loại thông tin	Nguồn	Cách sử dụng
System prompt	Mã backend	Quy định vai trò, ngôn ngữ, giới hạn và nguyên tắc gọi tool
Current message	Flutter request	Yêu cầu mới nhất cần xử lý

Chat history	Redis theo session	Giải quyết tham chiếu như “sân đó”, “giờ cũ” và hội thoại nhiều lượt
User context	JWT/database	user_id, role và business thuộc danh tính đã xác thực
Venue context	Lựa chọn UI/database	Giới hạn resource, menu, staff và giá đúng cơ sở
Agent scratchpad	AgentExecutor	Action và observation tạm thời trong một lượt
Tool result	Service/retriever	Dữ liệu thực tế để LLM tổng hợp câu trả lời
UI metadata	Response mapper	Card booking/order, điều hướng và trạng thái không cần parse từ văn bản

Bảng 3.38. Bảng thiết kế prompt, memory và ngữ cảnh tin cậy

Chat history và agent scratchpad là hai khái niệm khác nhau. History lưu các tin nhắn qua nhiều lượt và có thể tồn tại trong Redis theo TTL. Scratchpad chỉ lưu chuỗi action/observation của lần chạy hiện tại để LLM biết tool nào đã gọi và kết quả ra sao. Scratchpad không phải dữ liệu nghiệp vụ và không thay thế Booking, Order hoặc StaffRequest.

Prompt chỉ hướng dẫn hành vi, không phải hàng rào bảo mật. Những giá trị nhạy cảm như user_id, role, venue_id, store ownership và giá phải được lấy từ backend. Khi history quá dài, hệ thống cần giới hạn số message, tóm tắt hoặc loại bỏ nội dung cũ để không vượt context window và giảm chi phí.

3.5.4. Thiết kế Tool Registry và schema tham số

Tool	Tham số do LLM sinh	Xử lý thực tế
query_knowledge	query	Hybrid retrieval và graph expansion trên Neo4j
book_court	court_type, court_number, start_time, end_time, notes	BookingService kiểm tra resource/xung đột và tạo booking
check_schedule	date hoặc khoảng thời gian	Đọc lịch theo user/venue từ PostgreSQL
order_menu_items	items, notes	OrderService kiểm tra menu và tạo order transaction
recommend_menu	preference, limit	Truy vấn menu, tags và thống kê bán
call_staff	message, table_number, request_type	Tạo StaffRequest và notification realtime

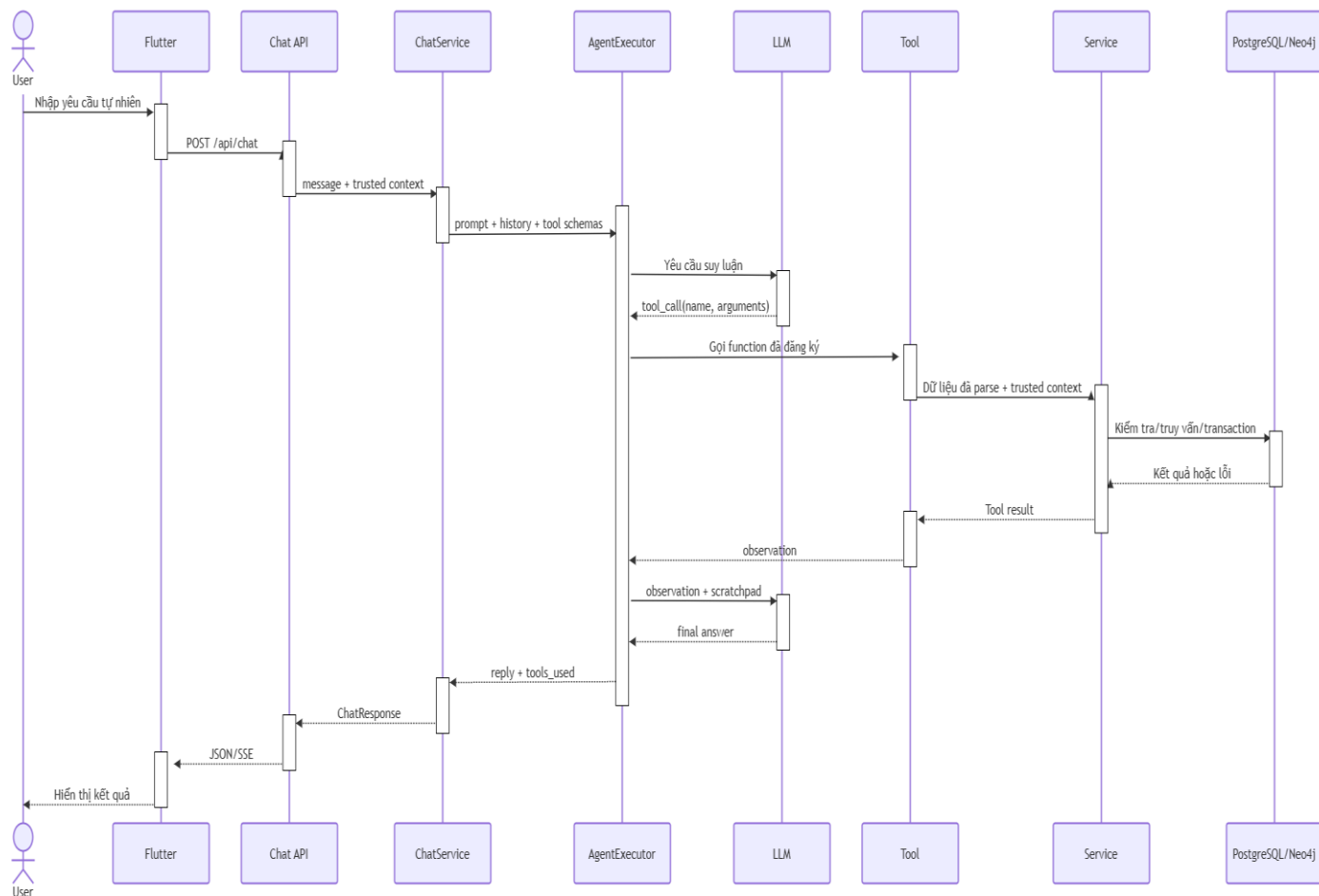
Bảng 3.39. Bảng thiết kế Tool Registry và schema tham số

Sinh tham số nghĩa là LLM chỉ tạo một đối tượng dữ liệu có cấu trúc phù hợp schema, ví dụ tên tool và các trường court_type/start_time. Đây chưa phải là thực thi. AgentExecutor nhận đối tượng đó, tìm Python function đã đăng ký, gọi function, nhận kết quả và gửi kết quả trở lại LLM. Business logic vẫn nằm trong service, không nằm trong LLM hoặc prompt.

Nguyên tắc thiết kế tool	Áp dụng
Một mục tiêu rõ ràng	Mỗi tool đại diện một khả năng như query, booking, order hoặc support
Schema chặt	Khai báo kiểu, trường bắt buộc, enum và mô tả để giảm arguments sai
Không lộ quyền nhạy cảm	Không nhận user_id/role tùy ý từ LLM
Kết quả có cấu trúc	Trả status, entity id, dữ liệu cần hiển thị và error code
Idempotency	Hành động thanh toán/ghi dữ liệu phải chống thực thi lặp khi retry
Timeout và giới hạn vòng lặp	Ngăn agent gọi tool vô hạn hoặc chờ dịch vụ ngoài quá lâu
Quan sát được	Ghi tool name, thời gian, kết quả và lỗi nhưng không log secret

Bảng 3.40. Các nguyên tắc thiết kế tool cho AI Agent

3.5.5. Luồng hoàn chỉnh của một Tool Calling



Hình 3.7. LLM Tool Calling sequence diagram

3.5.6. Cơ chế LangChain AgentExecutor và phân biệt Function/Tool Calling

Ở mức API mô hình, function calling là khả năng LLM sinh tên hàm và đối tượng arguments theo schema. Tool calling là khái niệm rộng hơn: tool có thể bọc một Python function, retriever, API, database adapter hoặc workflow. Trong hệ thống này, các Python function được đăng ký thành LangChain tools; vì vậy có thể nói LLM sinh một tool call, còn AgentExecutor ánh xạ tool đó đến function thực thi tương ứng.

- AgentExecutor gửi system prompt, history, message và danh sách tool schema đến LLM.
- LLM chọn trả lời trực tiếp hoặc sinh action gồm tool name và arguments.
- LangChain parse action, kiểm tra tool name và chuyển arguments sang kiểu của schema.

- Tool function được gọi trong backend; mọi trusted context được bổ sung ngoài arguments của LLM.
- Tool trả observation có cấu trúc hoặc lỗi đã chuẩn hóa.
- AgentExecutor thêm action/observation vào scratchpad và gọi lại LLM.
- LLM có thể gọi tool tiếp theo, hỏi bổ sung hoặc tạo final answer.
- Executor dừng khi có final answer, đạt giới hạn vòng lặp, timeout hoặc gặp lỗi không thể phục hồi.

Tình huống	Quyết định thiết kế
Trả lời chào hỏi/giải thích chung	Không gọi tool để giảm độ trễ
Hỏi dữ liệu cập nhật	Bắt buộc gọi tool tương ứng thay vì dựa vào trí nhớ LLM
Thiếu tham số bắt buộc	Hỏi lại người dùng, không tự đoán
Nhiều tool liên tiếp	Cho phép có giới hạn, observation của bước trước làm dữ liệu bước sau
Tool trả lỗi nghiệp vụ	LLM diễn giải lỗi và đề xuất lựa chọn hợp lệ
Arguments sai schema	Parser/validation từ chối và cho phép sửa trong giới hạn retry

Bảng 3.41. Chính sách điều phối và xử lý Tool Calling

3.5.7. Kỹ thuật truy hỏi Graph RAG

- Chuẩn hóa câu hỏi, phát hiện môn thể thao và loại nội dung được hỏi.
- Chạy full-text search trên name/description để lấy kết quả khớp từ và cụm từ.
- Sinh embedding câu hỏi và chạy vector search theo cosine similarity.
- Nếu index lỗi hoặc không có kết quả, dùng keyword CONTAINS làm fallback.
- Hợp nhất thứ hạng lexical và semantic bằng Reciprocal Rank Fusion.
- Tăng hạng theo đúng môn và loại node Rule/Technique/Equipment/Concept.
- Chọn seed node, duyệt các relationship được cho phép trong 1-2 bước.
- Đóng gói node, related entities, path và source thành context cho LLM.

Tiêu chí điểm	Ý nghĩa	Cách sử dụng
Full-text score	Độ khớp từ/cụm từ trong index	Xếp hạng lexical
Keyword match	Khớp trực tiếp tên/mô tả	Fallback và tăng độ chính xác tên riêng
Cosine similarity	Độ gần hướng giữa embedding câu hỏi và node	Xếp hạng semantic, không dùng đơn độc
RRF score	Tổng nghịch đảo vị trí từ nhiều danh sách	Hợp nhất rank không phụ thuộc scale
Sport boost	Node thuộc đúng môn được hỏi	Giảm lần tri thức giữa ba môn
Entity-type boost	Node đúng loại câu hỏi	Ưu tiên Rule cho câu luật, Technique cho câu kỹ thuật
Graph distance	Số cạnh từ seed đến node liên quan	Ưu tiên quan hệ gần và giải thích được

Bảng 3.42. Các tiêu chí xếp hạng kết quả truy xuất trong Graph RAG

3.5.8. Embedding, cosine và đồng bộ khởi động

Embedding biến name/description của mỗi KnowledgeEntity thành vector số. Cosine similarity đo độ gần về hướng giữa vector câu hỏi và vector node; giá trị cao hơn thường biểu thị nội dung gần nghĩa hơn. Cosine không bảo đảm câu trả lời đúng, đặc biệt khi model embedding chưa phù hợp tiếng Việt, nên hệ thống kết hợp lexical score, RRF, sport/type boost và quan hệ graph.

Khi backend khởi động, tác vụ nền bảo đảm vector index đúng dimension và chỉ đồng bộ node thiếu embedding hoặc đã stale. Log `checked=0 stored=0` nghĩa là không có node cần nhúng lại. Việc chạy nền tránh chặn startup; profile/model/hash nguồn được dùng để quyết định embedding còn hợp lệ.

3.5.9. Fallback, bảo mật và khả năng quan sát

Rủi ro	Cơ chế xử lý
LLM không khả dụng	Chuyển provider/fallback agent hoặc trả lỗi có kiểm soát
Hallucination	Bắt buộc tool cho dữ liệu động; Graph RAG cung cấp context; không có dữ liệu thì nói rõ
Prompt injection	Không trao quyền trực tiếp cho prompt; chỉ expose allowlist tools và kiểm tra tại service
Giả mạo danh tính	Trusted context lấy từ JWT/DB, không nhận user_id/role do LLM sinh
Tool loop	Giới hạn iteration, timeout và kích thước observation
Gọi lặp hành động ghi	Transaction, trạng thái hợp lệ và idempotency key khi phù hợp
Lộ dữ liệu nhạy cảm	Không đưa secret/password/camera credential vào prompt hoặc log
Neo4j/vector lỗi	Fallback full-text/keyword và trả nguồn dữ liệu còn khả dụng
Khó truy vết	Log request id, intent, tool name, latency, status và metadata không nhạy cảm

Bảng 3.43. Các rủi ro kỹ thuật và cơ chế xử lý

Response nên lưu hoặc trả `tools\_used`, intent, thời gian xử lý và mã lỗi chuẩn hóa để phục vụ kiểm thử. Không nên lưu toàn bộ chain-of-thought nội bộ của mô hình. Với mỗi tool, log cần phân biệt lỗi parse arguments, lỗi quyền, lỗi nghiệp vụ, lỗi hạ tầng và timeout để xác định đúng nguyên nhân.

3.5.10. Tiêu chí đánh giá hệ thống AI Agent

Nhóm đánh giá	Chỉ số đề xuất	Mục tiêu
Intent routing	Accuracy/F1 theo nhóm intent	Chọn đúng đường xử lý và đúng tool
Tool selection	Tool-name accuracy	Không gọi nhầm tool hoặc gọi tool khi không cần
Argument extraction	Field accuracy, valid-schema rate	Sinh đủ và đúng kiểu tham số
Task success	Tỷ lệ booking/order/request hoàn tất hợp lệ	Đánh giá kết quả nghiệp vụ end-to-end
Retrieval	Recall@k, MRR, nDCG và grounded answer rate	Đo chất lượng Graph RAG
Safety	Unauthorized-action rate, hallucination rate	Không vượt quyền và không bịa dữ liệu
Latency	P50/P95 tổng và theo tool	Đảm bảo trải nghiệm hội thoại
Reliability	Timeout/error/retry rate	Xác định độ ổn định của provider và hạ tầng
User experience	Tỷ lệ phải hỏi lại, số lượt hoàn thành tác vụ	Giảm hội thoại thừa

Bảng 3.44. Các chỉ số đánh giá hệ thống AI Agent

Bộ kiểm thử nên gồm câu đơn ý định, câu thiếu tham số, câu nhiều ý định, câu tham chiếu lịch sử, arguments không hợp lệ, người dùng sai quyền và lỗi dịch vụ ngoài. Đối với tri thức ba môn, cần xây tập câu hỏi chuẩn có node/path mong đợi để đánh giá retrieval độc lập với khả năng diễn đạt của LLM. Đây là cơ sở khoa học để chứng minh Agent được thiết kế hiệu quả thay vì chỉ đánh giá bằng một số đoạn hội thoại minh họa.

3.6. Thiết kế API và bảo mật

3.6.1. Các nhóm API chính

Nhóm API	Endpoint tiêu biểu	Mục đích
Auth	POST /api/auth/register, /login; POST /api/change-password	Tài khoản và token
Chat	POST /api/chat; POST /api/chat/stream	Chat thường và SSE
Venue	GET /api/venues; GET /venues/resources	Chọn cơ sở và tài nguyên
Booking	GET /api/availability; POST /api/booking; PATCH /cancel	Vòng đời booking
Menu/Order	GET /api/menu; POST /api/order	Menu và đơn venue
Staff Request	POST/GET/PATCH /api/staff/requests	Yêu cầu hỗ trợ
Realtime	GET/PATCH notifications; WebSocket	Thông báo và hội thoại realtime
Stripe	PaymentIntent, Checkout, webhook	Thanh toán test mode
Admin	/api/admin/*	Dashboard và quản trị nghiệp vụ
Partner	/api/partner/*	Store, partner menu và partner order
Camera	/api/admin/cameras; /api/staff/cameras	Quản lý và xem camera theo quyền
VNPay prototype	/api/payment/create, callback, query	Nghiên cứu tích hợp, chưa sandbox

Bảng 3.45. Các nhóm API chính của hệ thống

3.6.2. Xác thực, phân quyền và xử lý lỗi

- Mật khẩu được băm; request dùng JWT Bearer token và dependency xác thực.
- Role và venue/store scope được kiểm tra trước khi đọc hoặc thay đổi dữ liệu.
- user_id, role, venue_id được backend gắn vào tool context, không để LLM tự quyết định.
- Pydantic kiểm tra cấu trúc; service kiểm tra business rule và repository thực hiện transaction.

- Webhook Stripe phải xác thực chữ ký, xử lý idempotent và không tin hoàn toàn kết quả phía client.
- Lỗi Neo4j/vector được giảm cấp sang lexical retrieval; không có dữ liệu thì không trả câu trả lời.
- Lỗi LLM/tool được log và trả thông báo rõ, không để transaction ở trạng thái dở dang.
- Thông tin camera và khóa dịch vụ phải cấu hình bằng secret/env, không công khai trong response.

3.7. Đánh giá thiết kế

3.7.1. Ưu điểm

- Use case bao phủ Customer, Staff, Admin và Partner nhưng backend vẫn giữ kiến trúc monolith phù hợp phạm vi đồ án.
- LLM linh hoạt trong hiểu ngôn ngữ nhưng hành động bị giới hạn bởi tool schema và application service.
- PostgreSQL, Redis và Neo4j được sử dụng theo đặc tính dữ liệu thay vì thay thế lẫn nhau.
- Graph RAG kết hợp lexical, semantic và relationship context, tận dụng Neo4j tốt hơn tìm kiếm document thuần túy.
- Model, service, repository, retriever và tool có thể được kiểm thử độc lập.

3.7.2. Hạn chế và hướng hoàn thiện

- Chưa có benchmark retrieval định lượng đủ lớn cho câu hỏi tiếng Việt của ba môn.
- Chất lượng vector phụ thuộc model embedding; cosine score không được xem là bằng chứng đúng tuyệt đối.
- Graph expansion hiện chủ yếu 1-2 bước, chưa có reasoning theo path phức tạp hoặc reranker học máy.
- Một số chức năng cần dịch vụ ngoài, thiết bị thật và cấu hình môi trường để kiểm thử end-to-end.
- VNPay chưa có merchant code/hash secret nên chỉ là prototype nghiên cứu, không phải chức năng thanh toán hoạt động.

CHƯƠNG 4: GIAO DIỆN CHƯƠNG TRÌNH

4.1. Nguyên tắc thiết kế giao diện

4.1.1. Đối tượng sử dụng và điều hướng theo vai trò

Ứng dụng Flutter sử dụng cùng một codebase nhưng điều hướng khác nhau theo role. Customer ưu tiên chat, booking, menu và profile. Staff ưu tiên thông báo, yêu cầu hỗ trợ và inbox. Admin có dashboard và màn hình quản trị. Partner có shell riêng cho cửa hàng và đơn hàng.

4.1.2. Tính nhất quán và phản hồi trạng thái

Giao diện sử dụng màu sắc, khoảng cách, dialog, snackbar, loading và error widget dùng chung. Các thao tác bất đồng bộ phải thể hiện trạng thái đang xử lý, thành công hoặc thất bại; không để người dùng nhấn lặp tạo giao dịch. Metadata từ backend được dùng để dựng booking/order card thay vì phân tích ngược văn bản do LLM sinh.

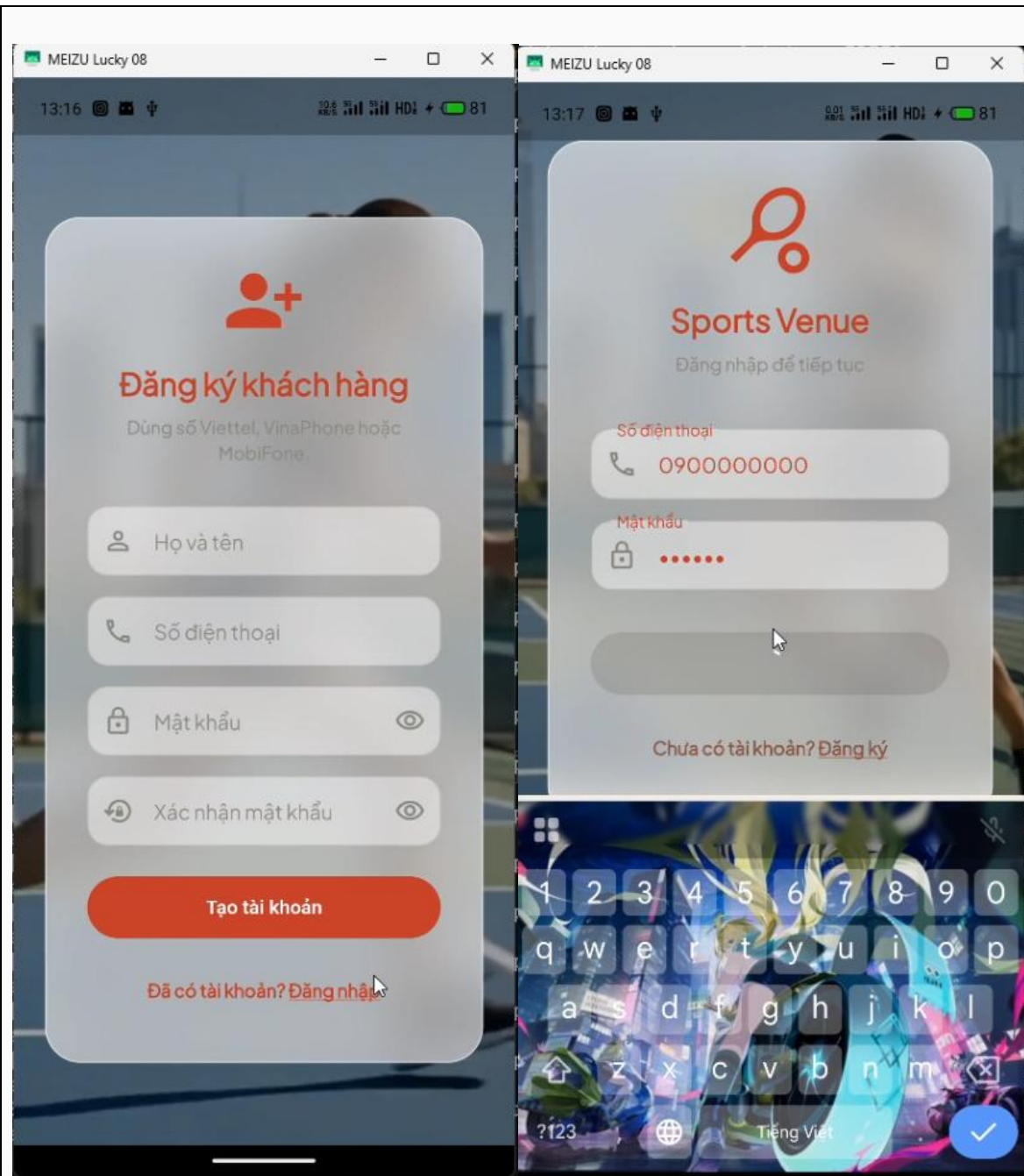
4.1.3. State management và kết nối backend

Riverpod quản lý auth state, chat state, booking, menu, payment và realtime notification. Dio gọi REST API; WebSocket duy trì chat và thông báo; SSE nhận phản hồi AI theo luồng. GoRouter điều hướng theo trạng thái đăng nhập và vai trò.

4.2. Giao diện khách hàng

4.2.1. Đăng nhập và màn hình chính

Màn hình đăng nhập nhận số điện thoại và mật khẩu, hiển thị lỗi xác thực rõ ràng. Sau đăng nhập, màn hình chính tập hợp lối tắt đến chat AI, đặt sân, dịch vụ, khám phá và hồ sơ. Venue context được giữ để các chức năng dùng đúng sân, menu và giá.



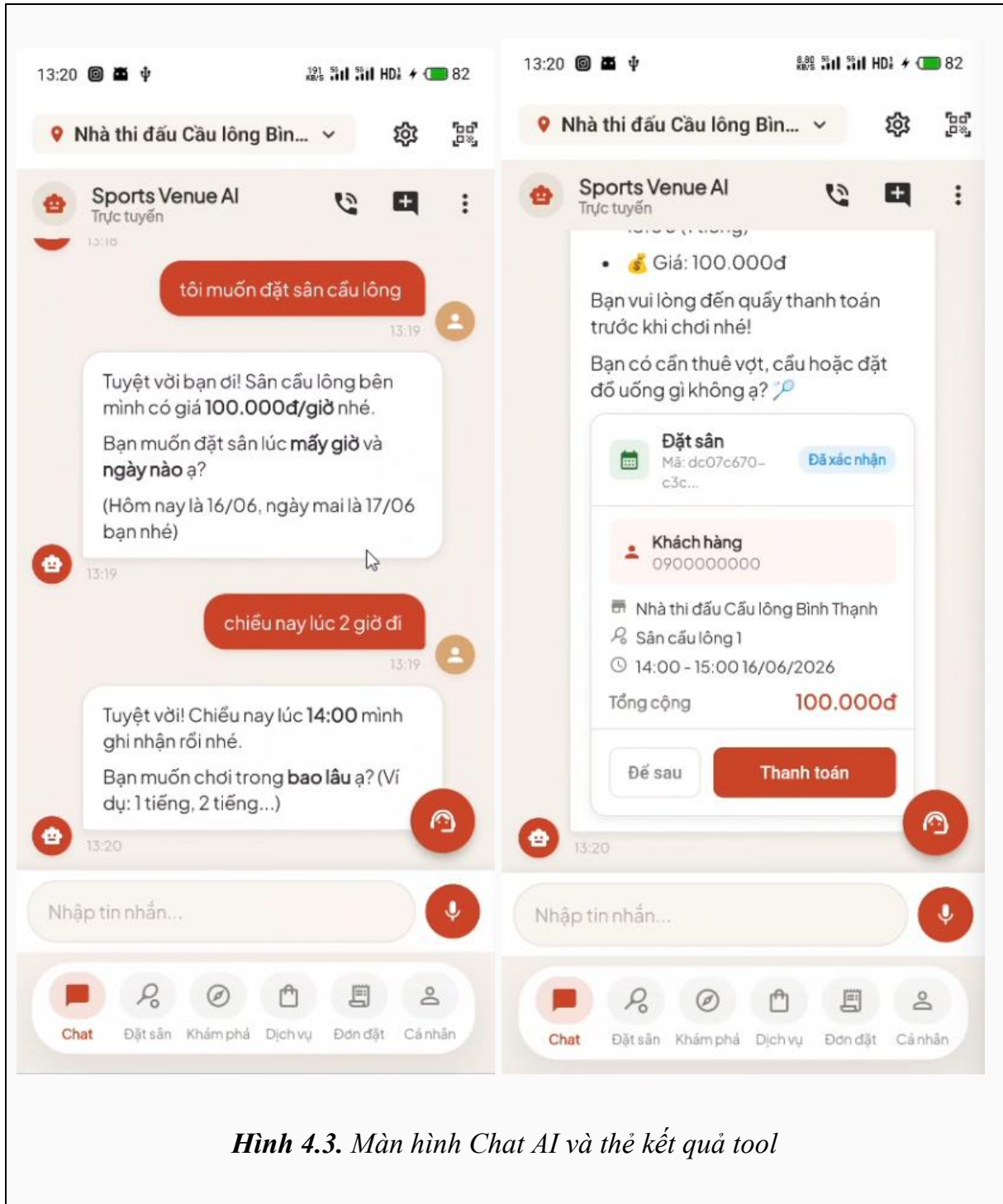
Hình 4.1. Màn hình đăng nhập



Hình 4.2. Màn hình chính của khách hàng

4.2.2. Giao diện Chat AI

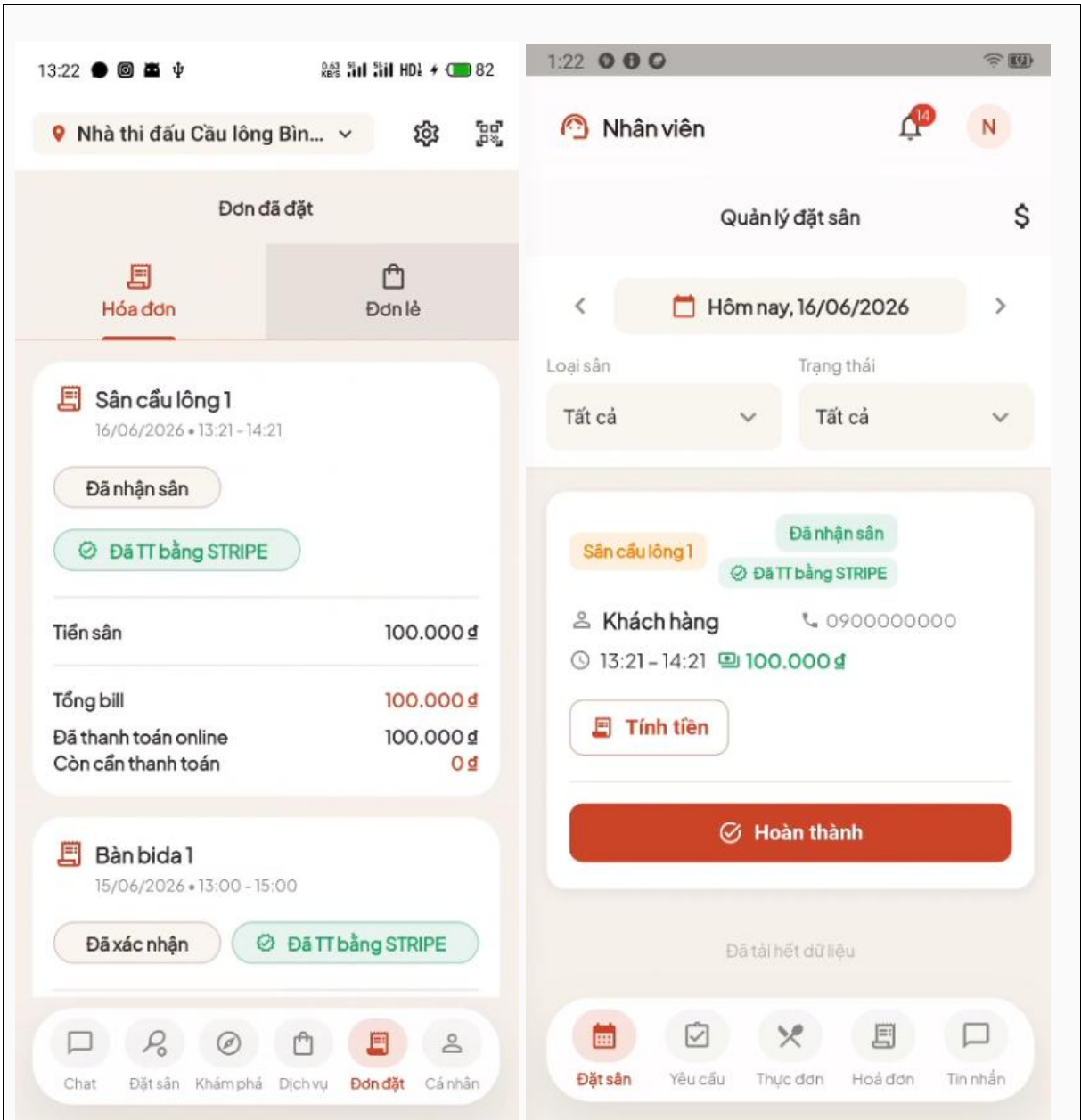
Màn hình chat hỗ trợ hội thoại nhiều lượt, markdown, streaming, voice input và TTS. Khi tool tạo booking hoặc order, phản hồi kèm card có dữ liệu cấu trúc và hành động tiếp theo. Người dùng không nhìn thấy JSON, tên tool hoặc agent scratchpad.



Hình 4.3. Màn hình Chat AI và thẻ kết quả tool

4.2.3. Đặt sân và lịch sử booking

Người dùng chọn ngày, loại resource, thời gian và sân/bàn. Ứng dụng gọi API availability trước khi tạo booking. Lịch sử hiển thị thời gian, trạng thái, giá và payment status; booking đang hoạt động được dùng làm context khi đặt món.

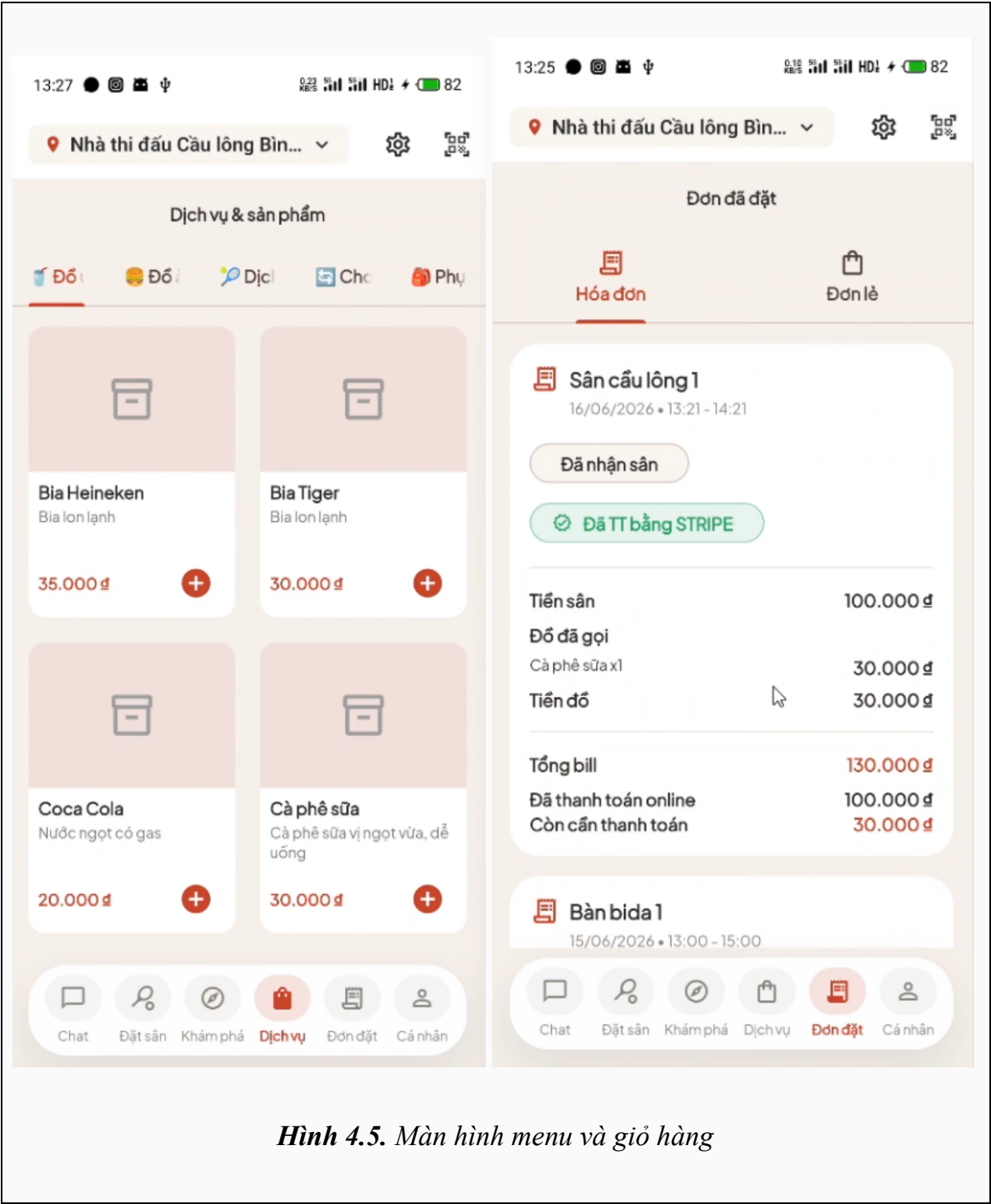


Hình 4.4. Màn hình đặt sân

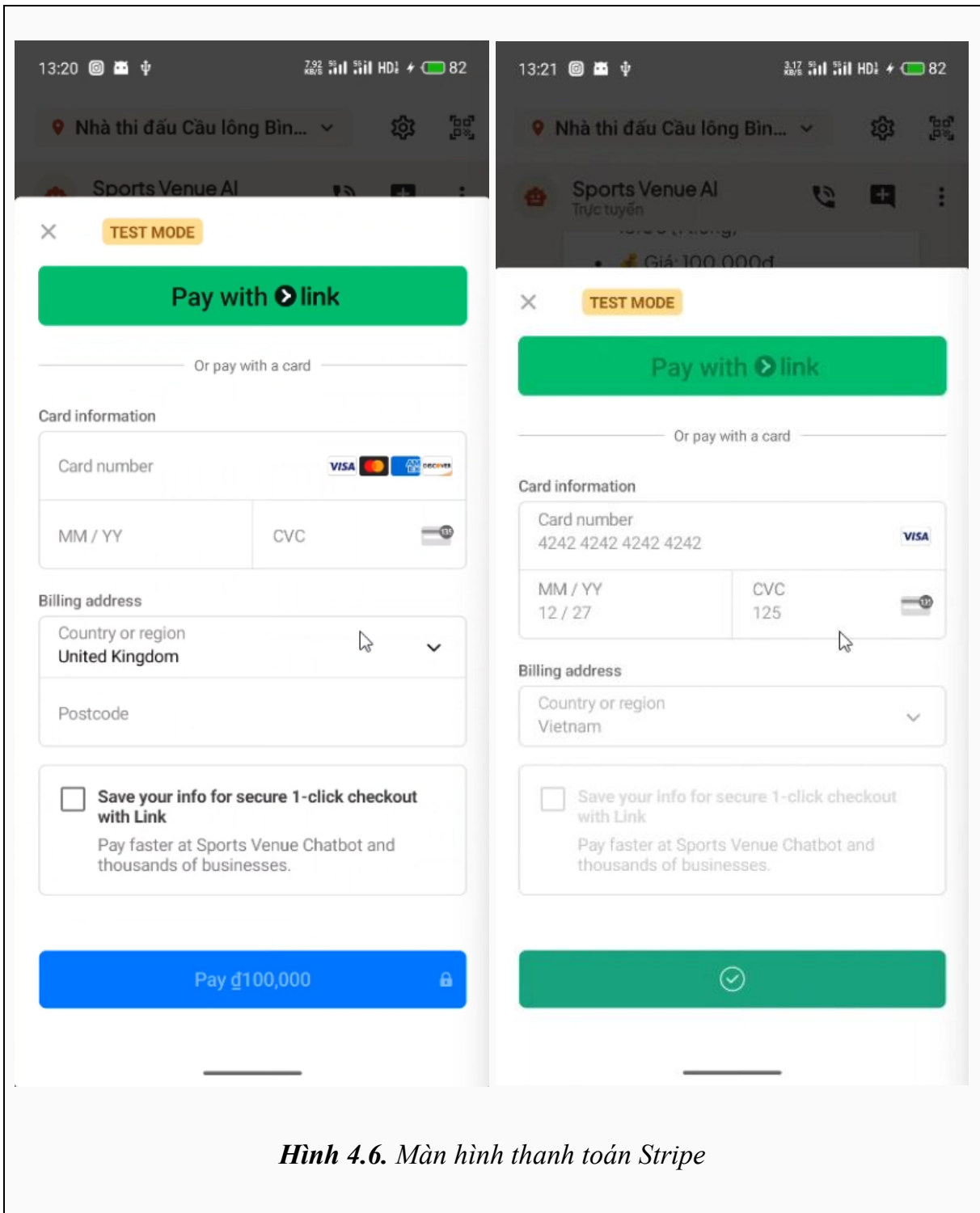
4.2.4. Menu, giỏ hàng và thanh toán

Menu được lọc theo venue và tình trạng phục vụ. Giỏ hàng tính số lượng, giá và tổng tiền trước khi tạo order. Stripe Payment Sheet được dùng trong môi trường thử

nghiệm. Nếu giao diện có tùy chọn VNPay, phải ghi rõ đây là prototype chưa khả dụng và không dẫn người dùng vào giao dịch giả.



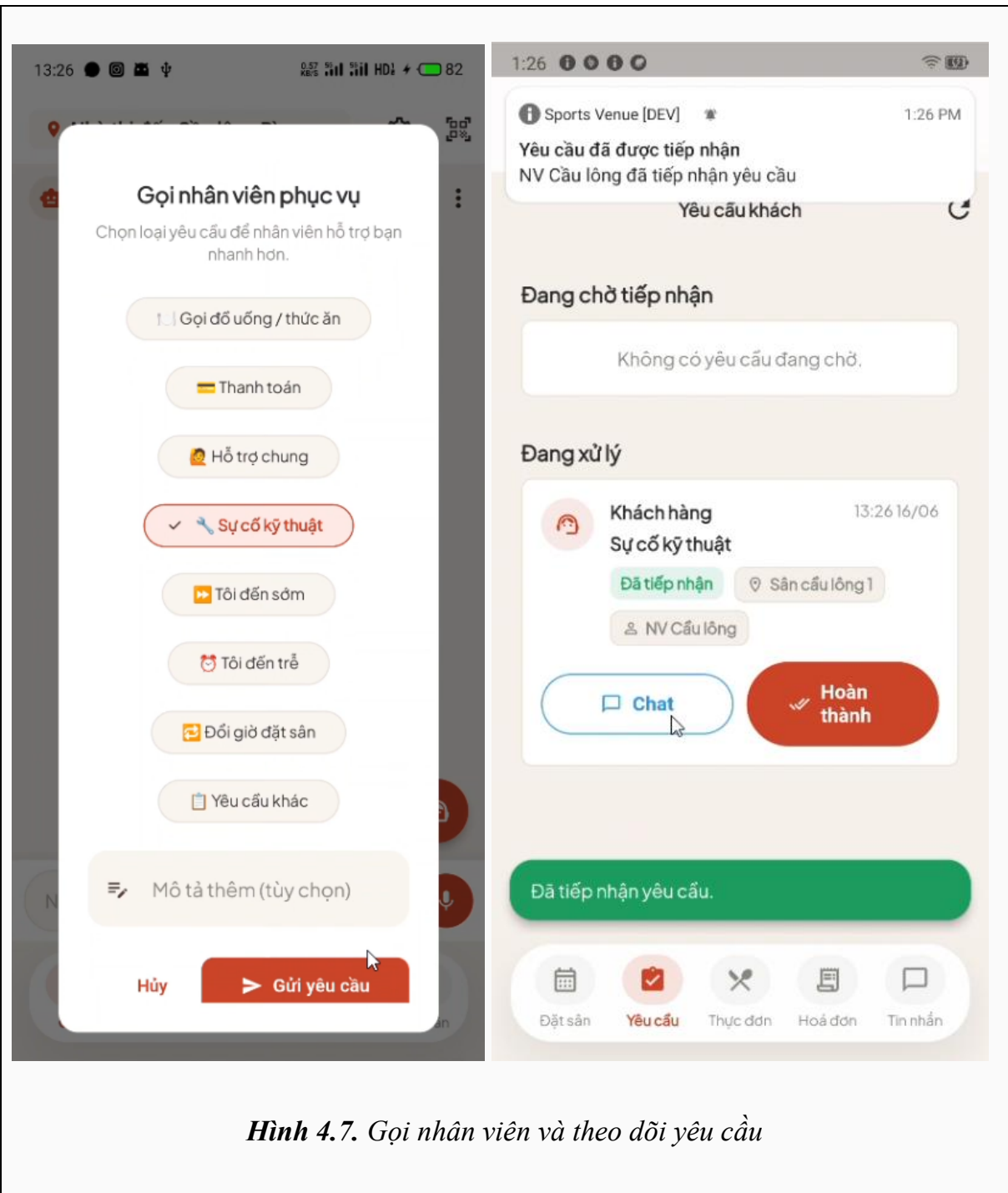
Hình 4.5. Màn hình menu và giỏ hàng



Hình 4.6. Màn hình thanh toán Stripe

4.2.5. Gọi nhân viên và chat realtime

Customer chọn loại yêu cầu, resource và mô tả. Backend tạo staff request, gửi thông báo đến nhân viên phù hợp và mở phòng chat khi cần. Trạng thái pending, accepted, completed hoặc cancelled được cập nhật realtime.

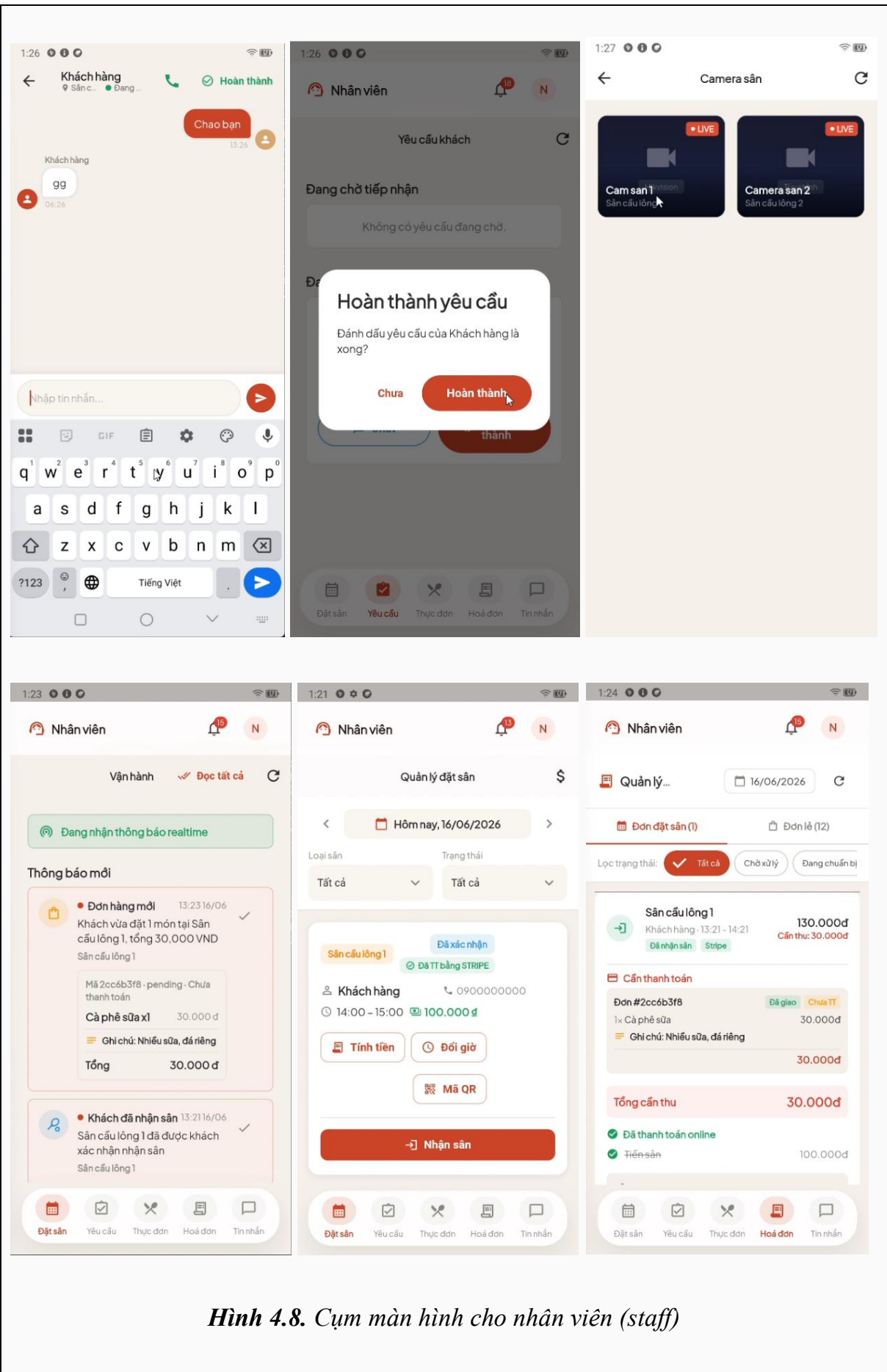


Hình 4.7. Gọi nhân viên và theo dõi yêu cầu

4.3. Giao diện nhân viên, quản trị và đối tác

4.3.1. Giao diện nhân viên

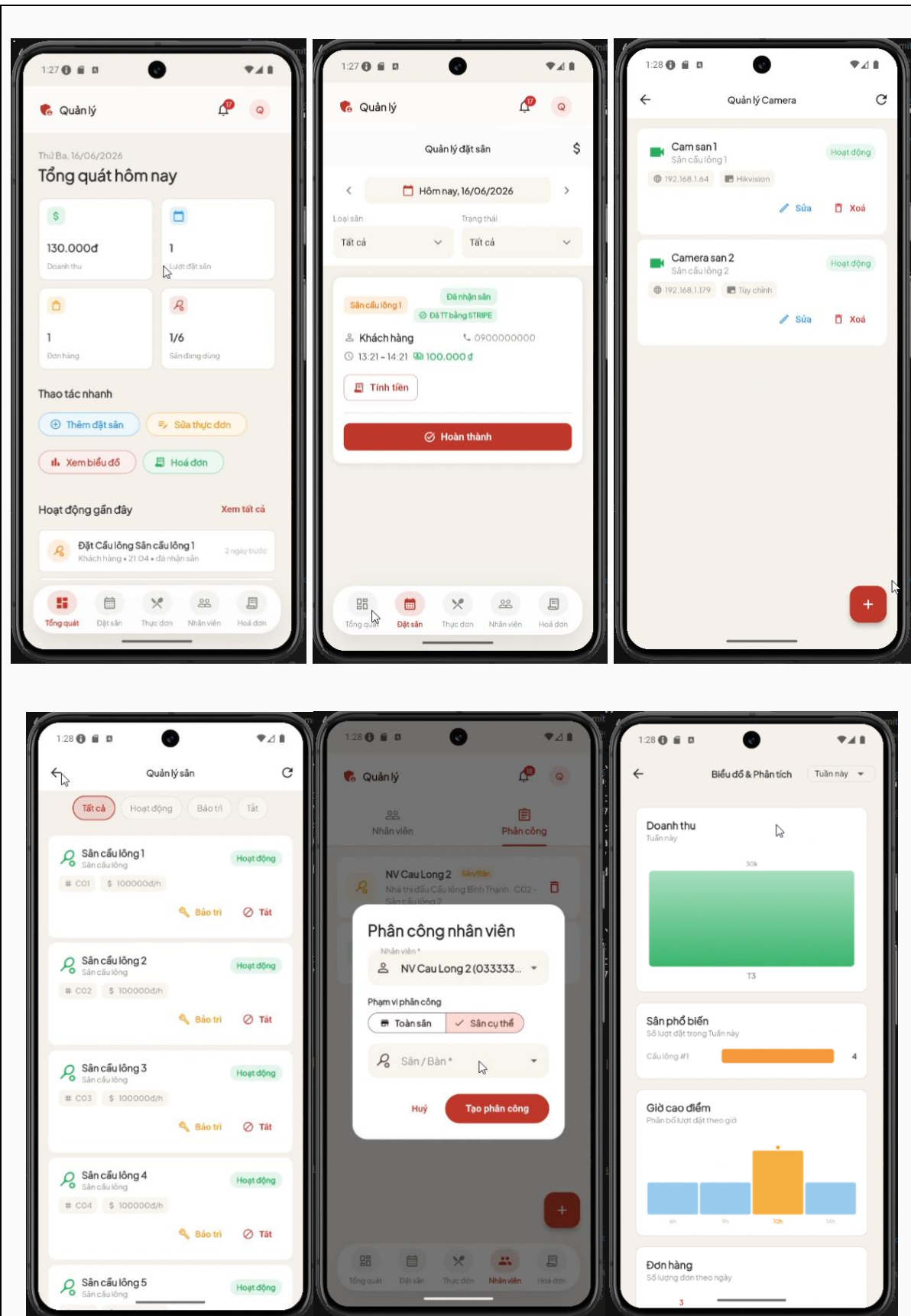
Staff shell tập trung vào thông báo vận hành, danh sách yêu cầu và inbox. Nhân viên chỉ xem dữ liệu trong phạm vi venue/resource được phân công. Màn hình camera là chức năng hỗ trợ và không làm thay đổi luồng nghiệp vụ chính.



Hình 4.8. Cùm màn hình cho nhân viên (staff)

4.3.2. Giao diện quản trị viên

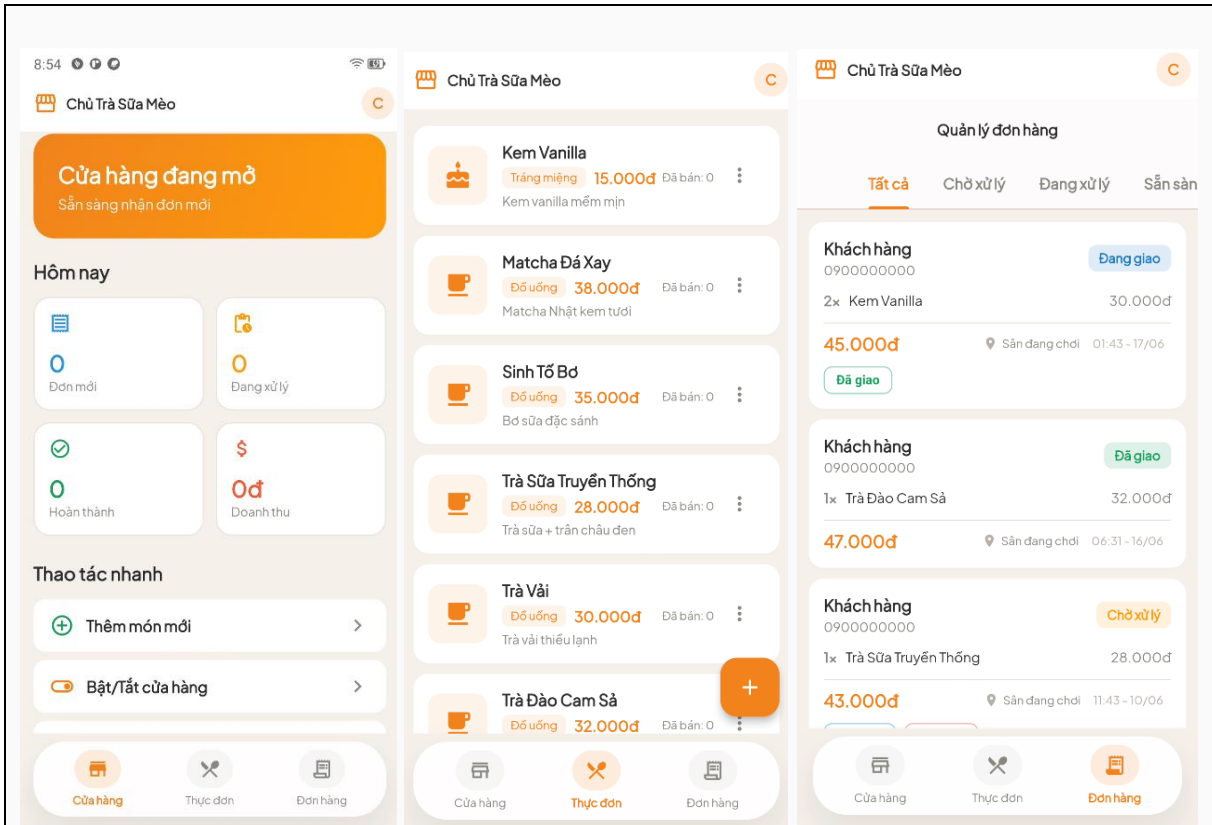
Admin quản lý booking, menu, resource, giá, nhân viên, hóa đơn và dashboard. Các màn hình dùng bộ lọc venue/trạng thái, bảng dữ liệu và dialog CRUD. Thao tác thay đổi giá hoặc trạng thái resource phải có xác nhận.



Hình 4.9. Cụm màn hình dashboard và quản trị (admin/quản lý)

4.3.3. Giao diện đối tác F&B

Partner shell gồm dashboard, menu và đơn hàng. Đối tác chỉ được thao tác cửa hàng thuộc tài khoản của mình. Vòng đời đơn đối tác gồm pending, accepted, preparing, ready, delivering, delivered hoặc cancelled.



Hình 4.10. Cụm màn hình dashboard và quản trị (partner/quản đối tác)

4.4. Môi trường chạy và cấu hình

4.4.1. Thành phần môi trường phát triển

Thành phần	Cấu hình sử dụng
Flutter App	Android/iOS; flavor appDev/appProd
FastAPI	Backend monolith tại localhost:8000 trong môi trường phát triển
PostgreSQL	Dữ liệu nghiệp vụ và transaction
Redis	Session, cache, presence, pub/sub
Neo4j Aura	Knowledge Graph, full-text và vector index
Ollama	LLM/embedding local tại localhost:11434
Stripe	Test mode và webhook
Java VNPay	Prototype gRPC; chưa có merchant credentials

Bảng 4.1. Môi trường triển khai hệ thống

4.4.2. Cấu hình khởi động AI và Knowledge Graph

Backend kết nối database, Redis và Neo4j trước khi khởi tạo ChatService. IntentRouter tải vector mẫu từ Redis hoặc sinh lần đầu. Sau khi ứng dụng startup hoàn tất, background task kiểm tra embedding KG. Log `checked=0 stored=0` nghĩa là dữ liệu đã cập nhật và không phát sinh nhúng lại.

4.4.3. Phạm vi Docker Compose

Docker Compose hiện chỉ chứa cấu hình phục vụ Java Payment Service của prototype VNPay. Backend FastAPI, PostgreSQL, Redis và Neo4j không được triển khai như một cụm Compose hoàn chỉnh. Vì vậy báo cáo xác định kiến trúc backend là monolith và không sử dụng thuật ngữ hệ thống microservices.

4.5. Kết quả thực nghiệm và kiểm thử

4.5.1. Kết quả chức năng

Nhóm	Kết quả
Chat AI	Hỗ trợ hội thoại, tool calling, streaming và metadata
Booking/Order	Tạo dữ liệu thật qua service/repository và kiểm tra điều kiện
Staff/Realtime	Yêu cầu, thông báo và chat WebSocket
Admin/Partner	Các luồng quản trị theo role và venue
Thanh toán	Stripe test mode; VNPay prototype chưa sandbox
Knowledge Graph	418 entity, 441 relationship và graph retrieval

Bảng 4.2. Kết quả chức năng

4.5.2. Kết quả Graph RAG

Pipeline đã lưu embedding cho 418/418 node. `entity\_embedding\_index` dùng cosine similarity, dimension 768 và ở trạng thái ONLINE. Runtime `query\_knowledge` đã nối full-text, vector search, RRF và graph expansion 1-2 bước. Khi vector hoặc index không khả dụng, truy vấn vẫn hoạt động ở mức lexical.

Kết quả smoke test cho thấy hạ tầng vector hoạt động nhưng chất lượng semantic tiếng Việt của `nomic-embed-text` chưa đủ tốt để dùng độc lập. Do đó kết quả đồ án được mô tả là Graph RAG hybrid đã hoạt động về cơ chế, chưa phải retrieval đã tối ưu hoặc đã đạt benchmark định lượng.

4.5.3. Kết quả kiểm thử phần mềm

Các test liên quan Knowledge Graph, incremental embedding và Redis cache của IntentRouter đã đạt. Lần chạy toàn bộ backend gần nhất đạt 85 test và còn 6 test lỗi ngoài phạm vi Graph RAG, chủ yếu do expectation intent cũ, mock Stripe và patch path của LangChain tool. Các lỗi này cần được xử lý trước khi coi bộ regression test đạt hoàn toàn.

4.5.4. Giới hạn thực nghiệm

- Chưa có tập câu hỏi chuẩn và chỉ số Precision@k, Recall@k, MRR hoặc nDCG.

- Chưa đánh giá tải đồng thời và độ trễ end-to-end trên môi trường production.
- Một số tính năng phụ thuộc thiết bị thật, camera, microphone hoặc dịch vụ ngoài.
- VNPay chưa có thông tin merchant nên chưa có bằng chứng sandbox end-to-end.
- Nội dung tri thức hiện giới hạn ở ba môn và chất lượng nguồn đã thu thập.

KẾT LUẬN VÀ KIẾN NGHỊ

Kết luận

Đồ án đã xây dựng một hệ thống chatbot AI cho quán thể thao với phạm vi liên kết ứng dụng di động, backend nghiệp vụ và truy vấn tri thức. Điểm trọng tâm là cơ chế Tool Calling: LLM sinh quyết định và tham số, trong khi tool, service và repository thực hiện nghiệp vụ thật. Thiết kế này giảm việc để mô hình tự suy diễn dữ liệu động hoặc thao tác trực tiếp database.

Knowledge Graph đã được nối vào runtime bằng hybrid retrieval, RRF và graph expansion. Embedding được đồng bộ tăng dần khi backend khởi động và cache giúp giảm request Ollama. Hệ thống sử dụng đúng lợi thế của Neo4j ở mức quan hệ lân cận, đồng thời nhận diện rõ giới hạn chất lượng embedding tiếng Việt.

Backend được xác định đúng là monolith phân tầng. Stripe là tích hợp thanh toán có thể kiểm thử; VNPay chỉ là prototype nghiên cứu do thiếu mã merchant và khóa bí mật. Việc mô tả đúng phạm vi giúp kết quả báo cáo phản ánh trung thực trạng thái mã nguồn.

Kiến nghị và hướng phát triển

- Xây dựng bộ benchmark retrieval tiếng Việt cho ba môn và đo Precision@k, Recall@k, MRR, nDCG.
- Thử nghiệm embedding đa ngôn ngữ như BGE-M3 trước khi điều chỉnh trọng số hybrid.
- Bổ sung graph path reasoning, source citation và kiểm tra độ tin cậy câu trả lời.
- Hoàn thiện authorization ở cấp tool/resource và tracing toàn bộ vòng lặp agent.
- Sửa các regression test còn lỗi và bổ sung kiểm thử end-to-end trên thiết bị thật.
- Nhận merchant credentials VNPay, kiểm thử sandbox, callback và đối soát trước khi đưa vào sử dụng.
- Triển khai môi trường staging, CI/CD, monitoring và đánh giá tải trước production.

TÀI LIỆU THAM KHẢO

- [1] FastAPI Documentation. Internet: <https://fastapi.tiangolo.com/>, truy cập ngày 27/04/2026.
- [2] Flutter Documentation. <https://docs.flutter.dev/>, truy cập ngày: 06/05/2026.
- [3] LangChain Documentation.
<https://docs.langchain.com/oss/python/langchain/overview>, truy cập ngày: 20/05/2026.
- [4] Neo4j Documentation. <https://neo4j.com/docs/>, truy cập ngày: 20/05/2026.
- [5] Ollama, nomic-embed-text model. <https://ollama.com/library/nomic-embed-text>, truy cập ngày: 30/05/2026.
- [6] PostgreSQL Documentation. <https://www.postgresql.org/docs/current/index.html>, truy cập ngày: 27/04/2026.
- [7] Redis Documentation. <https://redis.io/docs/latest/>, truy cập ngày: 27/05/2026.
- [8] Stripe Documentation. <https://docs.stripe.com/>, truy cập ngày: 13/05/2026.
- [9] VNPay Sandbox Documentation. <https://sandbox.vnpayment.vn/apis/>, truy cập ngày: 13/05/2026.
- [10] World Pool-Billiard Association. <https://wpapool.com/>, truy cập ngày: 21/05/2026
- [11] USA Pickleball Official Rules. <https://usapickleball.org/docs/rules/USAP-Official-Rulebook.pdf>, truy cập ngày: 21/05/2026
- [12] Badminton World Federation. <https://bwfbadminton.com/>, truy cập ngày: 21/05/2026
- [13] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, Douwe Kiela. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Internet: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf, truy cập ngày 31/05/2026
- [14] RTSP Cameras,
https://docs.openedgeplatform.intel.com/2026.0/edge-ai-libraries/dlstreamer-pipeline-server/advanced-guide/detailed_usage/camera/rtsp.html, truy cập ngày 10/06/2026
- [15] Wechat Mini Program guide,
<https://developers.weixin.qq.com/miniprogram/en/dev/framework/>, truy cập ngày 13/06/2026

- [16] Gordon V. Cormack, Charles L.A. Clarke, Stefan Butt University of Waterloo, 2009. *Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods*. Internet: <https://dl.acm.org/doi/10.1145/1571941.1572114>, truy cập ngày 15/06/2026
- [17] SQLAlchemy Documentation. <https://docs.sqlalchemy.org/> , truy cập ngày: 27/04/2026.
- [18] Riverpod Documentation. https://riverpod.dev/docs/introduction/getting_started , truy cập ngày: 06/05/2026.
- [19] M. Jones, J. Bradley, N. Sakimura. RFC 7519: *JSON Web Token (JWT)*. Internet: <https://datatracker.ietf.org/doc/html/rfc7519> , truy cập ngày: 10/05/2026.